🧠 AI Evolution Program

Parte 02 — IA probabilística, evolutiva y de decisión

Introduce decisiones bajo incertidumbre, modelos temporales, simulación, lógica difusa y métodos de búsqueda inspirados en la naturaleza.

1. 025 — Razonamiento con incertidumbre
2. 026 — Teorema de Bayes y actualización de creencias
3. 027 — Redes bayesianas e independencia condicional
4. 028 — Modelos ocultos de Markov
5. 029 — Procesos de decisión de Markov
6. 030 — Teoría de decisión y utilidad esperada
7. 031 — Métodos Monte Carlo y simulación
8. 032 — Lógica difusa y control aproximado
9. 033 — Algoritmos genéticos
10. 034 — Optimización por enjambre y colonia
11. 035 — Programación probabilística y causalidad
12. 036 — Proyecto: sistema híbrido para decisiones

025 — Razonamiento con incertidumbre

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **razonamiento con incertidumbre** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar razonamiento con incertidumbre usando los conceptos `incertidumbre`, `evidencia`, `creencias`, `riesgo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`incertidumbre`, `evidencia`, `creencias`, `riesgo`

🗺️ Ubicación en el mapa de la IA

La IA simbólica de la Parte 01 asume conocimiento completo y correcto: si la base de hechos dice `llueve`, entonces llueve. El mundo real no coopera: los sensores fallan, los datos son parciales y las reglas tienen excepciones. El razonamiento con incertidumbre reemplaza la lógica booleana por la teoría de la probabilidad como "lógica extendida" para grados de creencia. Es el fundamento de todo lo que sigue en esta parte: Bayes (026), redes bayesianas (027), HMM (028) y MDP (029) son formas cada vez más estructuradas de gestionar la misma pregunta: ¿qué debo creer y hacer cuando no lo sé todo?

📖 Fundamentos

🎲 Por qué la lógica no basta

Considera la regla lógica `dolor_de_muelas → caries`. Es falsa (puede ser gingivitis), y su inversa `caries → dolor_de_muelas` también (hay caries asintomáticas). Enumerar todas las excepciones es inviable (**pereza**), la ciencia no las conoce todas (**ignorancia teórica**) y no podemos examinar cada caso (**ignorancia práctica**). La probabilidad resume esas tres fuentes de incertidumbre en un número: `P(caries |`

`dolor_de_muelas) = 0.6` no afirma que la caries sea "60 % verdadera", sino que, dado lo que sabemos, asignamos un grado de creencia de 0.6.

📐 El lenguaje: variables aleatorias y distribuciones

- **Variable aleatoria**: función del espacio muestral a valores. `Clima ∈ {sol, lluvia, nube, nieve}`.
- **Distribución conjunta** $P(X_1, \ldots, X_n)$: asigna probabilidad a cada combinación de valores. Es el objeto que contiene *toda* la información probabilística del dominio.
- **Axiomas de Kolmogórov**: $0 \le P(a) \le 1$, $P(\text{verdad}) = 1$, y para eventos disjuntos $P(a \vee b) = P(a) + P(b)$. De ellos se deriva todo lo demás, incluida la regla general $P(a \vee b) = P(a) + P(b) - P(a \wedge b)$.

🔧 Las tres operaciones básicas

Dada la conjunta, todo se responde con tres mecanismos:

```
1. Marginalización:      P(X) = Σ_y P(X, y)
   ("sumar fuera" las variables que no interesan)

2. Condicionamiento:     P(X | e) = P(X, e) / P(e)
   (restringir el universo a los mundos donde la evidencia e es cierta
    y renormalizar)

3. Regla del producto:   P(X, Y) = P(X | Y) · P(Y)
   (encadenada n veces da la regla de la cadena:
    P(X₁,…,Xₙ) = Π_i P(Xᵢ | X₁,…,Xᵢ₋₁))
```

La **inferencia por enumeración** responde cualquier consulta `P(Q | e)` recorriendo la tabla conjunta: filtra las filas compatibles con `e`, suma por valores de `Q` y normaliza. Es exacta pero cuesta `O(dⁿ)` en tiempo y memoria para `n` variables con `d` valores: por eso existen las redes bayesianas (clase 027).

🔗 Independencia: el arma contra la explosión combinatoria

- $X \perp Y$ (independencia): $P(X, Y) = P(X) \cdot P(Y)$. Rara en la práctica.
- $X \perp Y \mid Z$ (independencia condicional): $P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z)$. Muy común: el dolor de muelas y que el gancho del dentista se enganche son dependientes entre sí, pero independientes *dado* que hay caries (la causa común explica ambos).

Una conjunta de 32 variables binarias tiene $2^{32} - 1 \approx 4.3 \times 10^9$ parámetros libres; si las variables son condicionalmente independientes dada una causa común, bastan 65. La independencia condicional es la razón de ser de los modelos gráficos.

⚖️ Creencias, evidencia y riesgo

- **Creencia (belief)**: distribución `P(H)` sobre hipótesis, previa a observar.
- **Evidencia**: valor observado de algunas variables; condiciona la creencia.
- **Riesgo**: la decisión no depende solo de `P(H | e)` sino del **costo del error**. Con `P(caries|e) = 0.4`, tratar o no tratar depende de la utilidad de cada resultado (esto se formaliza en la clase 030). Un agente racional maximiza utilidad esperada, no probabilidad de acierto.

🧮 Ejemplo trabajado

Dominio de 3 variables binarias: `Caries`, `Dolor`, `Engancha` (el gancho del dentista se engancha). Distribución conjunta (suma 1.000):

Caries	Dolor	Engancha	P
sí	sí	sí	0.108
sí	sí	no	0.012
sí	no	sí	0.072
sí	no	no	0.008
no	sí	sí	0.016
no	sí	no	0.064
no	no	sí	0.144
no	no	no	0.576

Consulta 1 — marginal: `P(Caries=sí) = 0.108+0.012+0.072+0.008 = 0.200`.

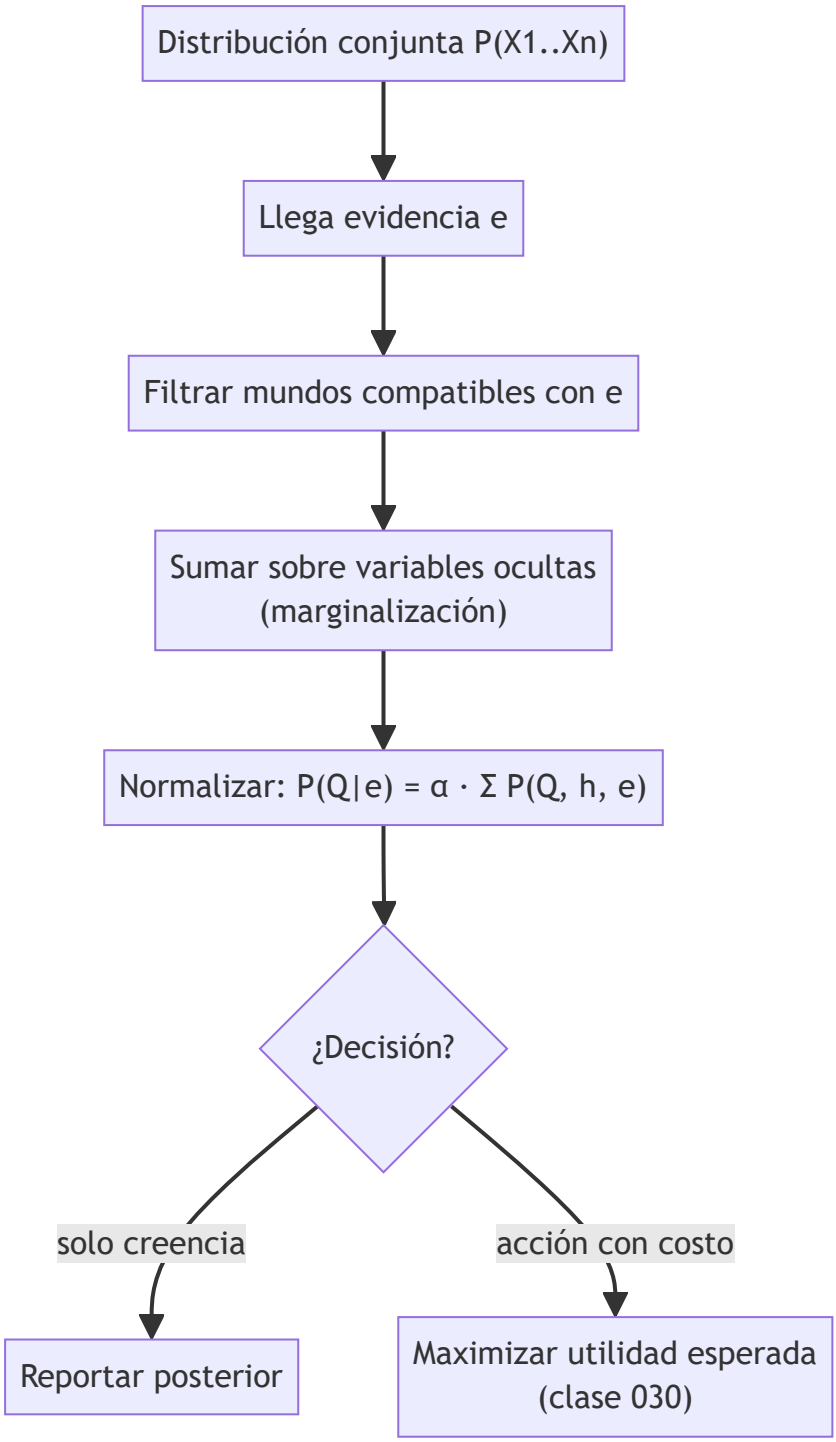
Consulta 2 — condicional: `P(Caries=sí | Dolor=sí)`. Filas con `Dolor=sí`: `P(Dolor=sí) = 0.108+0.012+0.016+0.064 = 0.200`. De ellas, con caries: `0.108+0.012 = 0.120`. Resultado: `0.120 / 0.200 = 0.600`. El dolor triplica la creencia (0.2 → 0.6).

Consulta 3 — más evidencia: `P(Caries=sí | Dolor=sí, Engancha=sí) = 0.108 / (0.108+0.016) = 0.871`. Cada pieza de evidencia coherente refuerza la creencia, pero nunca llega a certeza.

Verificación de independencia condicional: `P(Engancha=sí | Caries=sí) = (0.108+0.072)/0.200 = 0.9`, y `P(Engancha=sí | Caries=sí, Dolor=sí) = 0.108/0.120 = 0.9`. Dado `Caries`, el dolor no aporta nada sobre el gancho: `Engancha ⊥ Dolor | Caries`.

📊 Propiedades y comparación

Enfoque	Representa incertidumbre	Combina evidencia	Costo	Limitación principal
Lógica proposicional	No (verdadero/falso)	Conjunción monótona	Bajo	Colapsa ante excepciones
Lógica no monótona / default	Cualitativa	Retracción de conclusiones	Medio	Sin grados: no distingue 0.51 de 0.99
Factores de certeza (MYCIN)	Numérica ad hoc	Fórmulas heurísticas	Bajo	Incoherente al encadenar evidencia
Teoría de probabilidad	Grados de creencia [0,1]	Condicionamiento (Bayes)	Alto en la conjunta plena	Exige estimar parámetros
Conjuntos difusos	Vaguedad de predicados	Operadores min/max	Bajo	Modela imprecisión, no ignorancia (clase 032)



⚠️ Errores conceptuales frecuentes

1. **"Probabilidad 0.6 significa que el hecho es parcialmente verdadero."** Falso: el mundo está en un estado definido; el 0.6 mide *nuestra* creencia dada la evidencia disponible. Es epistemología, no ontología.
2. **Confundir `P(a|b)` con `P(b|a)`.** `P(dolor|caries) = 0.6` no implica `P(caries|dolor) = 0.6`; la inversión exige el teorema de Bayes con los priors (clase 026).
3. **Ignorar la normalización.** `P(Caries=sí, Dolor=sí) = 0.12` no es la respuesta a "¿probabilidad de caries dado dolor?"; hay que dividir por `P(Dolor=sí)`.

4. **Suponer independencia sin justificarla.** Multiplicar `P(a)·P(b)` cuando `a` y `b` comparten causa produce estimaciones drásticamente erróneas (el error clásico de "naive" mal aplicado).
5. **Tratar la creencia como decisión.** Elegir la hipótesis más probable ignora costos asimétricos: con `P(incendio) = 0.02` no se ignora la alarma, porque el costo del falso negativo es enorme.

🚀 Del aprendizaje a la operación

El laboratorio manipula una conjunta diminuta y perfectamente conocida. En un sistema real: (1) la conjunta nunca se conoce — se estima desde datos con error de muestreo; (2) la enumeración es inviable a partir de decenas de variables y se necesitan modelos gráficos o inferencia aproximada; (3) las probabilidades deben *calibrarse* (que un 0.7 declarado acierte ~70 % de las veces) y monitorearse ante deriva de datos; (4) la salida probabilística exige una capa de decisión con costos explícitos y revisión humana proporcional al riesgo.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4.ª ed., cap. 12 "Quantifying Uncertainty". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Jaynes, E. T. (2003). *Probability Theory: The Logic of Science*. Cambridge University Press. — uso: desarrollo extendido del tema
- Koller, D. & Friedman, N. (2009). *Probabilistic Graphical Models*, caps. 2-3. https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/ — uso: referencia consultada en su fuente original
- Murphy, K. (2022). *Probabilistic Machine Learning: An Introduction*, cap. 2. https://probml.github.io/pml-book/book1.html — uso: desarrollo extendido del tema
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P88 · Probabilidad, frecuencia y expectativa razonable	1946	Demuestra que la probabilidad no es una convención entre varias: es la única forma consistente de extender la lógica a grados de creencia.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	citada en las referencias de esta clase · caps. 2-3 · obra de referencia de la parte 02
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	citada en las referencias de esta clase · cap. 2
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	citada en las referencias de esta clase · obra de referencia de la parte 02
Jaynes, E. T. — *Probability Theory: The Logic of Science*	2003	*sin localizador verificado*	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 12 · obra de referencia de la parte 02

📝 Evaluación completa

❓ Preguntas

1. Define razonamiento con incertidumbre sin usar una marca o framework como definición.
2. Explica la relación entre incertidumbre, evidencia, creencias, riesgo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

026 — Teorema de Bayes y actualización de creencias

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **teorema de bayes y actualización de creencias** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar teorema de bayes y actualización de creencias usando los conceptos `Bayes`, `prior`, `likelihood`, `posterior`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`Bayes`, `prior`, `likelihood`, `posterior`

🗺️ Ubicación en el mapa de la IA

La clase anterior estableció la probabilidad como lenguaje de creencias; esta clase da el **mecanismo de actualización**: el teorema de Bayes convierte creencias previas más evidencia en creencias posteriores, de forma matemáticamente obligada (no opcional). Es el corazón operativo de la IA probabilística: el clasificador naive Bayes, el filtrado en HMM (028), la inferencia en redes bayesianas (027) y la programación probabilística (035) son, en el fondo, aplicaciones repetidas de esta única ecuación de 1763.

📖 Fundamentos

📜 El teorema

De la regla del producto `P(a ∧ b) = P(a|b)P(b) = P(b|a)P(a)` se despeja:

```
              P(e | H) · P(H)
P(H | e) = ──────────────────
                   P(e)

P(H)     prior       — creencia antes de ver la evidencia
P(e|H)   likelihood  — qué tan compatible es la evidencia con la hipótesis
P(e)     evidencia   — constante de normalización: Σ_h P(e|h)P(h)
P(H|e)   posterior   — creencia actualizada
```

Su valor práctico: solemos conocer la dirección **causal** `P(síntoma | enfermedad)` (estable, medible en estudios) y necesitamos la dirección **diagnóstica** `P(enfermedad | síntoma)` (la que cambia con cada epidemia, porque depende del prior). Bayes hace la inversión con contabilidad correcta.

🔁 Actualización secuencial

Con evidencias $e_1, e_2, \ldots$ el posterior de hoy es el prior de mañana:

$$P(H \mid e_1, e_2) \propto P(e_2 \mid H, e_1) \cdot P(H \mid e_1)$$

Si las evidencias son condicionalmente independientes dado `H` (supuesto *naive Bayes*), el término se simplifica a $P(e_2|H)$ y la actualización es un producto de likelihoods sobre el prior. El orden de llegada de la evidencia no altera el posterior final.

🧻 Forma en odds: la versión mental

Para hipótesis binarias conviene trabajar con momios (odds) $O(H) = P(H)/P(\neg H)$:

$$O(H \mid e) = O(H) \cdot LR \qquad \text{donde} \quad LR = P(e|H) \,/\, P(e|\neg H)$$

El **factor de Bayes / razón de verosimilitud (LR)** mide cuánta fuerza probatoria tiene la evidencia: LR = 1 no informa; LR = 10 multiplica los momios por 10. Esta forma evita la falacia de la tasa base porque obliga a partir de los momios previos.

🧪 Estimación y suavizado

Cuando los parámetros se estiman de datos, la frecuencia cruda `count(x)/N` asigna probabilidad 0 a lo no visto, y un solo cero aniquila cualquier producto de likelihoods. El **suavizado de Laplace** añade pseudo-conteos: $P(x) = (\text{count}(x)+\alpha)/(N+\alpha k)$ con k valores posibles. Es, en términos bayesianos, un prior Dirichlet sobre los parámetros.

🧠 Interpretación

- **Frecuentista**: la probabilidad es límite de frecuencias en repeticiones. Bien definida solo para eventos repetibles.
- **Bayesiana**: la probabilidad es grado de creencia coherente; permite hablar de `P(hipótesis)`. El teorema de Cox y el argumento *Dutch book* muestran que cualquier sistema de creencias graduadas que evite incoherencias debe obedecer los axiomas de probabilidad.

🧮 Ejemplo trabajado

Test diagnóstico. Prevalencia `P(D) = 0.01`, sensibilidad `P(+|D) = 0.99`, especificidad `P(−|¬D) = 0.95` (es decir, tasa de falsos positivos 0.05). Llega un resultado positivo.

```
P(+) = P(+|D)P(D) + P(+|¬D)P(¬D)
     = 0.99·0.01  + 0.05·0.99
     = 0.0099     + 0.0495     = 0.0594

P(D|+) = 0.0099 / 0.0594 = 0.1667  ≈ 1/6
```

Con un test "99 % sensible", el positivo solo implica ~17 % de probabilidad de enfermedad: de 10 000 personas, 99 enfermas dan positivo verdadero pero 495 sanas dan falso positivo. **Segundo test positivo independiente** (posterior como nuevo prior):

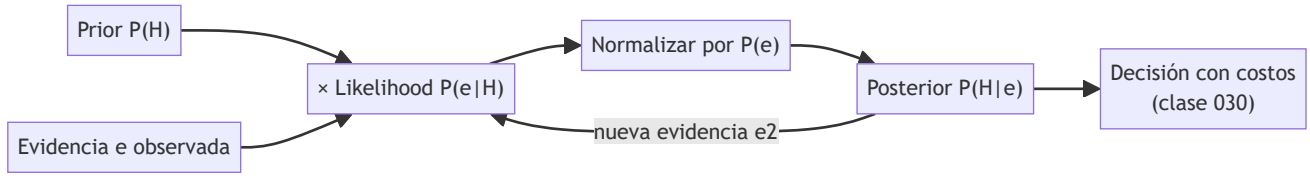
```
O(D|+) = (0.1667/0.8333) = 0.2 ;  LR = 0.99/0.05 = 19.8
O(D|++) = 0.2 · 19.8 = 3.96  →  P(D|++) = 3.96/4.96 = 0.798
```

Dos positivos elevan la creencia a ~80 %. La tabla resume la trayectoria de la creencia:

Evidencia acumulada	P(D)
ninguna (prior)	0.010
un positivo	0.167
dos positivos	0.798
tres positivos	0.987

📊 Propiedades y comparación

Propiedad	Actualización bayesiana	Estimación frecuentista (MLE)	Reglas ad hoc (scores)
Usa conocimiento previo	Sí (prior explícito)	No	Implícito y opaco
Coherencia al encadenar evidencia	Garantizada	N/A	No garantizada
Datos pequeños	Robusta (prior regulariza)	Sobreajusta / ceros	Frágil
Costo	Necesita likelihood y prior	Solo datos	Bajo
Crítica principal	Subjetividad del prior	Ignora la tasa base	Sin semántica



⚠️ Errores conceptuales frecuentes

1. **Falacia de la tasa base**: interpretar `P(+|D) = 0.99` como `P(D|+) ≈ 0.99`. El ejemplo trabajado muestra que con prevalencia 1 % el posterior es ~0.17.
2. **Falacia del fiscal**: confundir `P(evidencia | inocente)` pequeña con `P(inocente | evidencia)` pequeña; ignora cuántos inocentes hay en la población de referencia.
3. **Contar dos veces la evidencia correlacionada**: multiplicar likelihoods de dos tests que comparten mecanismo de error como si fueran independientes infla el posterior.
4. **Prior 0 o 1**: asignar probabilidad exacta 0 a una hipótesis la hace irrecuperable ante cualquier evidencia (con `P(H)=0`, el posterior es 0 para siempre). Regla práctica de Cromwell: reservar 0/1 para lógica, no para creencias empíricas.
5. **"El prior es trampa subjetiva"**: no declararlo no lo elimina; el MLE equivale a un prior uniforme implícito. La honestidad está en declararlo y hacer análisis de sensibilidad.

🚀 Del aprendizaje a la operación

Aquí los parámetros (prevalencia, sensibilidad) se dan como conocidos; en operación se estiman con intervalos de confianza y caducan (la prevalencia cambia con el tiempo y el lugar). Un sistema real necesita: recalibración periódica del prior con datos frescos, verificación de la independencia asumida entre fuentes de evidencia, suavizado ante eventos no vistos y umbrales de decisión derivados de costos clínicos o de negocio, no del 0.5 por defecto.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Bayes, T. & Price, R. (1763). "An Essay towards solving a Problem in the Doctrine of Chances". *Phil. Trans. Royal Society*, 53, 370-418. https://doi.org/10.1098/rstl.1763.0053 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 12.5-12.6. https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Murphy, K. (2022). *Probabilistic Machine Learning: An Introduction*, cap. 4 (estadística bayesiana). https://probml.github.io/pml-book/book1.html — uso: desarrollo extendido del tema
- Gelman, A. et al. (2013). *Bayesian Data Analysis*, 3.ª ed. http://www.stat.columbia.edu/~gelman/book/ — uso: referencia consultada en su fuente original

- Jurafsky, D. & Martin, J. H. *Speech and Language Processing*, 3.ª ed. (draft), cap. de naive Bayes. https://web.stanford.edu/~jurafsky/slp3/ — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P87 · Ensayo para resolver un problema en la doctrina de las probabilidades	1763	La regla que invierte el condicional: pasar de «qué esperaría ver si la hipótesis fuese cierta» a «cuán probable es la hipótesis dado lo que he visto».	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase
Murphy, Kevin P. — *Probabilistic Machine Learning*	2022	ISBN 9780262046824 · web de la obra	citada en las referencias de esta clase · cap. 4
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 12 · obra de referencia de la parte 02
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define teorema de bayes y actualización de creencias sin usar una marca o framework como definición.
2. Explica la relación entre Bayes, prior, likelihood, posterior.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

027 — Redes bayesianas e independencia condicional

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **redes bayesianas e independencia condicional** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar redes bayesianas e independencia condicional usando los conceptos `DAG`, `inferencia`, `independencia`, `factores`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`DAG`, `inferencia`, `independencia`, `factores`

🗺️ Ubicación en el mapa de la IA

La conjunta plena (clase 025) es exacta pero exponencial; Bayes (026) actualiza creencias pero no dice cómo representar dominios grandes. Las redes bayesianas (Pearl, años 80) resuelven ambos problemas: codifican la conjunta como un grafo dirigido acíclico (DAG) que explota independencias condicionales. Fueron el estándar del razonamiento incierto en IA durante dos décadas y son la base de los HMM (028), de la programación probabilística (035) y —leídas causalmente— del do-calculus de Pearl.

📖 Fundamentos

Definición

Una **red bayesiana** es un par (G, Θ):

- **G**: DAG cuyos nodos son variables aleatorias; una arista `X → Y` expresa dependencia directa (a menudo causal).
- **Θ**: para cada nodo, una **tabla de probabilidad condicional (CPT)** `P(Xᵢ | Padres(Xᵢ))`.

La red define la conjunta por la **regla de la cadena factorizada**:

```
P(X₁, …, Xₙ) = Π_i P(Xᵢ | Padres(Xᵢ))
```

Ahorro: con `n` variables binarias y a lo sumo `k` padres por nodo, los parámetros pasan de `2ⁿ − 1` a `≤ n·2ᵏ`. Para n = 30, k = 3: de ~10^9 a 240.

🧭 Semántica local: cada variable es independiente de sus no-descendientes dados sus padres

Esa es la única suposición que hace la red. Todo lo demás (qué independencias se cumplen entre pares arbitrarios) se lee del grafo con **d-separación**.

🔀 d-separación: las tres conexiones elementales

Un camino entre `X` e `Y` queda **bloqueado** por el conjunto observado `Z` según el tipo de tramo:

```
1. Cadena     X → M → Y   bloqueado si M ∈ Z   (mediador observado)
2. Bifurcación X ← M → Y  bloqueado si M ∈ Z   (causa común observada)
3. Colisionador X → M ← Y bloqueado si M ∉ Z y ningún descendiente de M ∈ Z
                          (¡observar el colisionador ABRE el camino!)
```

`X ⊥ Y | Z` se garantiza si **todos** los caminos entre `X` e `Y` están bloqueados por `Z`. El caso 3 produce el fenómeno de **explaining away**: dos causas independientes de un mismo efecto se vuelven dependientes al observar el efecto (si la alarma suena y hubo terremoto, baja la creencia en robo).

⚙️ Inferencia

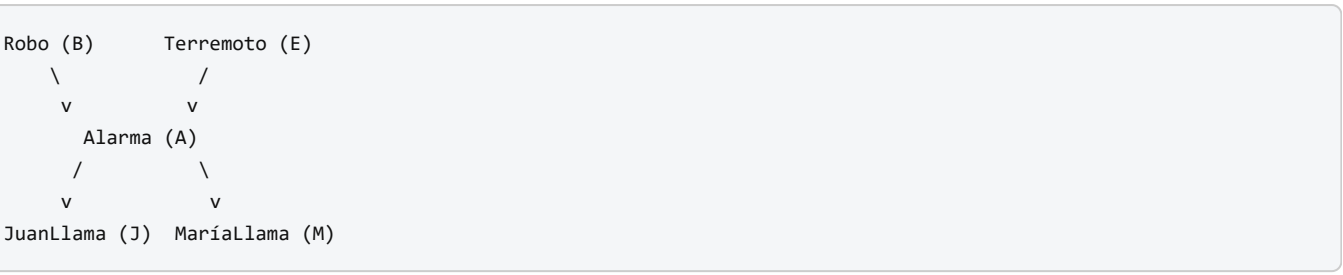
- **Enumeración**: expandir `P(Q|e) = α Σ_h Π_i P(xᵢ|padres)` sobre las variables ocultas `h`. Exponencial en el número de ocultas.
- **Eliminación de variables**: intercalar sumas y productos, guardando resultados intermedios como **factores**; el costo lo domina el factor más grande generado (ancho del orden de eliminación). Exacta y mucho más eficiente en la práctica.
- La inferencia exacta en redes arbitrarias es **NP-dura**; en **politrees** (a lo sumo un camino no dirigido entre cada par) es lineal. Para redes densas se usa inferencia aproximada por muestreo (clase 031).

📐 Construcción

Orden recomendado: elegir variables → ordenarlas (causas antes que efectos) → para cada una, elegir el conjunto mínimo de padres que la haga independiente de las anteriores → llenar CPTs. Un orden anticausal produce redes correctas pero densas y con parámetros antinaturales.

🧮 Ejemplo trabajado

Red clásica de Pearl (alarma antirrobo):



CPTs: $P(B)=0.001$, $P(E)=0.002$; $P(A|B,E)=0.95$, $P(A|B,\neg E)=0.94$, $P(A|\neg B,E)=0.29$, $P(A|\neg B,\neg E)=0.001$; $P(J|A)=0.90$, $P(J|\neg A)=0.05$; $P(M|A)=0.70$, $P(M|\neg A)=0.01$.

Conjunta de un mundo concreto (los cinco factores se multiplican):

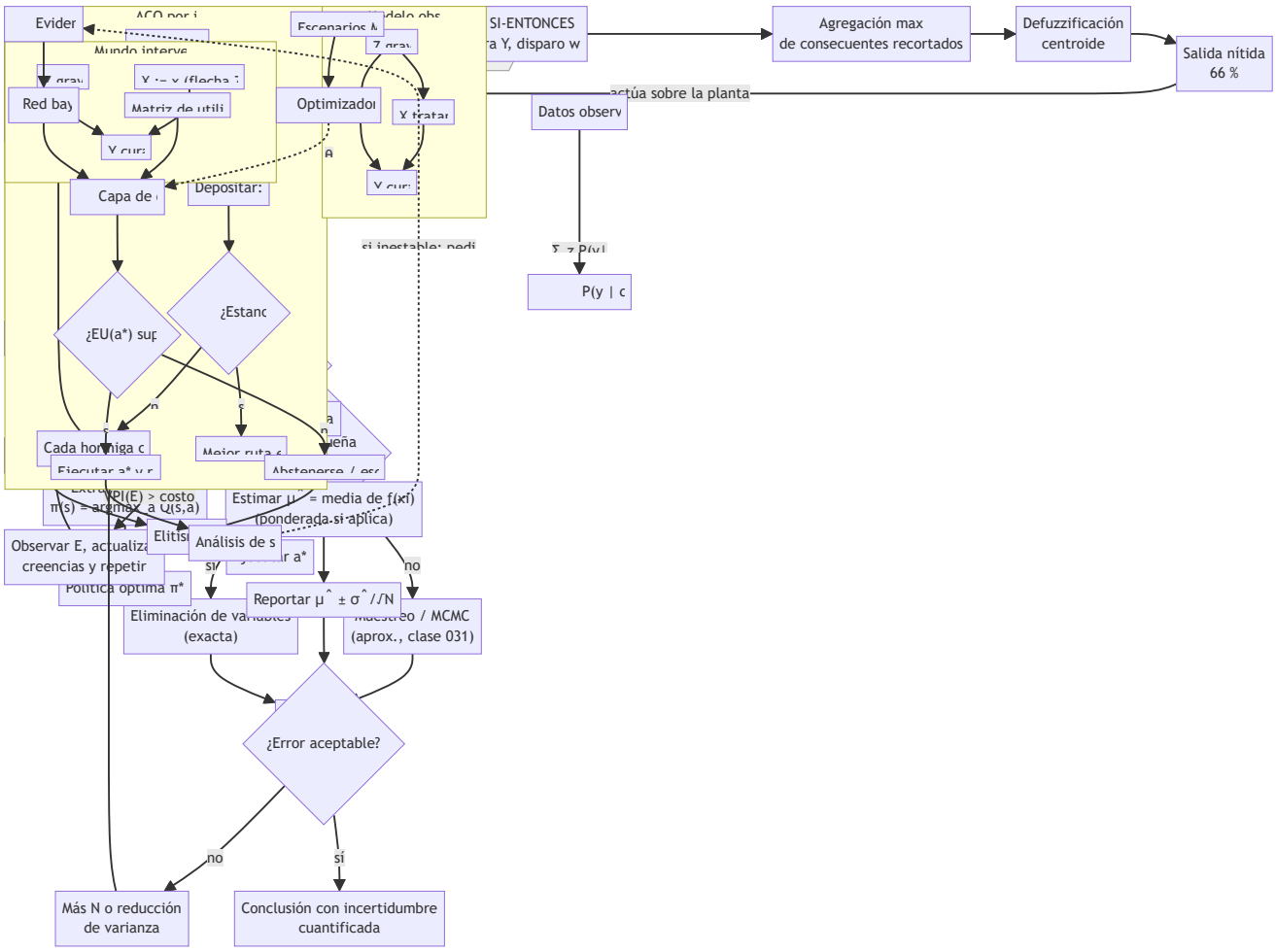
$$\begin{aligned} P(j, m, a, \neg b, \neg e) &= P(j|a)\cdot P(m|a)\cdot P(a|\neg b,\neg e)\cdot P(\neg b)\cdot P(\neg e) \\ &= 0.90 \cdot 0.70 \cdot 0.001 \cdot 0.999 \cdot 0.998 \\ &= 0.000628 \end{aligned}$$

Consulta diagnóstica $P(B \mid j, m)$: enumerando las 8 combinaciones de $\{A, E\}$ para $B=b$ y $B=\neg b$ se obtiene $P(B|j,m) = \alpha\cdot\langle 0.00059224, 0.0014919\rangle \approx \langle 0.284, 0.716\rangle$. Aunque ambos vecinos llamaron, la probabilidad de robo es solo **28.4 %**: el prior 0.001 pesa mucho (compárese con la falacia de tasa base de la clase 026).

d-separación en la red: $J \perp M \mid A$ (bifurcación con A observada: las llamadas solo se correlacionan a través de la alarma); $B \perp E$ a priori, pero $B \not\perp E \mid A$ (colisionador observado: explaining away).

📊 Propiedades y comparación

Representación	Parámetros (n binarias)	Inferencia exacta	Lee independencias	Interpretación causal
Conjunta plena	$2^n - 1$	$O(2^n)$	No (implícitas)	No
Naive Bayes	$2n + 1$	$O(n)$	Solo una (todo $\perp$ dado la clase)	No
Red bayesiana	$\leq n \cdot 2^k$	NP-dura (lineal en politree)	d-separación	Opcional (si aristas = mecanismos)
Red de Markov (no dirigida)	según cliques	NP-dura	separación simple	No



⚠️ Errores conceptuales frecuentes

1. **"Arista = causalidad garantizada."** La red solo codifica independencias; la lectura causal exige supuestos extra (clase 035). Dos DAG distintos pueden representar la misma distribución (equivalencia de Markov: `X→Y` y `X←Y` son indistinguibles sin más datos).
2. **"Ausencia de arista = variables independientes."** Significa ausencia de dependencia *directa*; puede haber dependencia mediada por otros caminos.
3. **Tratar el colisionador como una cadena.** Condicionar en un efecto común *crea* dependencia (sesgo de selección): en un hospital, enfermedades independientes en la población parecen correlacionadas entre ingresados.
4. **Creer que la inferencia siempre escala.** Es NP-dura en general; el ancho de árbol de la red decide si la exacta es viable.
5. **Llenar CPTs con frecuencias crudas de pocos datos.** Produce ceros estructurales que anulan mundos posibles; se necesita suavizado o priors (clase 026).

🚀 Del aprendizaje a la operación

El laboratorio usa una red diminuta con CPTs dadas. En producción: la **estructura** se aprende de datos (búsqueda con score BIC o tests de independencia) o se elicita de expertos con sesgos documentados; los **parámetros** requieren estimación con intervalos; la inferencia en redes grandes exige motores dedicados

(variable elimination optimizada, junction tree o muestreo); y hay que monitorear que las independencias asumidas sigan siendo válidas cuando el proceso generador cambia.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann. — uso: desarrollo extendido del tema
- Koller, D. & Friedman, N. (2009). *Probabilistic Graphical Models: Principles and Techniques*, caps. 3-9. https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/ — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 13 "Probabilistic Reasoning". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Cooper, G. F. (1990). "The computational complexity of probabilistic inference using Bayesian belief networks". *Artificial Intelligence*, 42(2-3), 393-405. https://doi.org/10.1016/0004-3702(90)90060-D — uso: fuente primaria del mecanismo estudiado
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*, 2.ª ed., cap. 1. https://bayes.cs.ucla.edu/BOOK-2K/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P91 · Fusión, propagación y estructuración en redes de creencia	1986	Hace tratable la probabilidad en IA: la estructura del grafo dice qué hay que almacenar y qué se puede propagar localmente.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	citada en las referencias de esta clase · caps. 3-9 · obra de referencia de la parte 02
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	citada en las referencias de esta clase · obra de referencia de la parte 02
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 13 · obra de referencia de la parte 02

📝 Evaluación completa

❓ Preguntas

1. Define redes bayesianas e independencia condicional sin usar una marca o framework como definición.
2. Explica la relación entre DAG, inferencia, independencia, factores.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;

- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

028 — Modelos ocultos de Markov

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **modelos ocultos de markov** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelos ocultos de markov usando los conceptos `HMM`, `transición`, `emisión`, `Viterbi`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`HMM`, `transición`, `emisión`, `Viterbi`

🗺️ Ubicación en el mapa de la IA

Las redes bayesianas (027) modelan un instante; los HMM añaden el **tiempo**: son una red bayesiana dinámica desenrollada, con un estado oculto que evoluciona como cadena de Markov y observaciones ruidosas de ese estado. Dominaron el reconocimiento de voz, el etiquetado gramatical y la bioinformática durante ~30 años (tutorial de Rabiner, 1989), y sus algoritmos (forward, Viterbi) reaparecen en los MDP (029), en el filtrado de Kalman y, conceptualmente, en cualquier sistema que mantiene una "creencia de estado" — incluidos los agentes modernos.

📖 Fundamentos

🧩 Definición

Un HMM discreto es λ = (S, V, A, B, π):

```
S = {s₁…s_N}   estados ocultos          V = {v₁…v_M}  símbolos observables
A: a_ij = P(q_{t+1}=s_j | q_t=s_i)      matriz de transición  (N×N)
B: b_j(k) = P(o_t=v_k | q_t=s_j)        matriz de emisión     (N×M)
π: π_i = P(q₁=s_i)                       distribución inicial
```

Dos supuestos estructurales: **Markov de orden 1** (el estado siguiente depende solo del actual) y **emisión condicionada solo al estado actual**. La conjunta se factoriza:

```
P(q₁…q_T, o₁…o_T) = π_{q₁} b_{q₁}(o₁) · Π_{t=2}^{T} a_{q_{t-1} q_t} b_{q_t}(o_t)
```

❓ Los tres problemas de Rabiner

1. **Evaluación** — `P(O | λ)`: ¿qué tan probable es la secuencia observada? → algoritmo **forward**.
2. **Decodificación** — `argmax_Q P(Q | O, λ)`: ¿cuál es la secuencia de estados más probable? → algoritmo de **Viterbi**.
3. **Aprendizaje** — ajustar λ para maximizar `P(O | λ)` → **Baum-Welch** (EM sobre HMM).

➡️ Forward (evaluación en $O(N^2T)$ en lugar de $O(N^T)$)

La variable forward `α_t(i) = P(o₁…o_t, q_t = s_i)` se calcula por programación dinámica:

```
Inicialización:  α₁(i) = π_i · b_i(o₁)
Recursión:       α_{t+1}(j) = [ Σ_i α_t(i) · a_ij ] · b_j(o_{t+1})
Terminación:     P(O|λ) = Σ_i α_T(i)
```

Normalizando `α_t` en cada paso se obtiene el **filtrado**: `P(q_t | o₁…o_t)`, la creencia actual del estado. La variable backward `β_t(i)` permite además el **suavizado** `P(q_t | o₁…o_T)` (revisar el pasado con información futura).

🏆 Viterbi (decodificación)

Igual recursión pero con `max` en lugar de `Σ`, guardando punteros al mejor predecesor:

```
δ₁(i) = π_i b_i(o₁)
δ_{t+1}(j) = max_i [ δ_t(i) · a_ij ] · b_j(o_{t+1});   ψ_{t+1}(j) = argmax_i …
Camino: q*_T = argmax_i δ_T(i), luego retroceder por ψ.
```

Complejidad $O(N^2T)$. En la práctica se trabaja con logaritmos para evitar underflow (productos de cientos de probabilidades < 1).

🔁 Baum-Welch (esbozo)

EM: con λ actual se calculan las responsabilidades `γ_t(i)` (estar en `i` en `t`) y `ξ_t(i,j)` (transitar `i→j` en `t`) usando forward-backward; luego se re-estiman A, B, π como frecuencias esperadas. Garantiza no disminuir `P(O|λ)`; converge a un óptimo local.

🧮 Ejemplo trabajado

HMM del clima con 2 estados ocultos {`Lluvia` (R), `Sol` (S)} y observación {`paraguas` (u), `sin paraguas` (n)}:

```
π = (0.5, 0.5)
A: R→R 0.7, R→S 0.3, S→R 0.3, S→S 0.7
B: b_R(u)=0.9, b_R(n)=0.1, b_S(u)=0.2, b_S(n)=0.8
```

Observaciones: `O = (u, u, n)`.

Forward:

```
t=1: α₁(R)=0.5·0.9=0.45          α₁(S)=0.5·0.2=0.10
t=2: α₂(R)=(0.45·0.7+0.10·0.3)·0.9=(0.315+0.030)·0.9=0.3105
     α₂(S)=(0.45·0.3+0.10·0.7)·0.2=(0.135+0.070)·0.2=0.0410
t=3: α₃(R)=(0.3105·0.7+0.0410·0.3)·0.1=(0.21735+0.0123)·0.1=0.022965
     α₃(S)=(0.3105·0.3+0.0410·0.7)·0.8=(0.09315+0.0287)·0.8=0.097480
P(O|λ)=0.022965+0.097480=0.120445
```

Filtrado en t=3: `P(R|uun) = 0.022965/0.120445 ≈ 0.191` — el día sin paraguas desploma la creencia en lluvia.

Viterbi:

```
t=1: δ₁(R)=0.45, δ₁(S)=0.10
t=2: δ₂(R)=max(0.45·0.7, 0.10·0.3)·0.9 = 0.315·0.9 = 0.2835   (desde R)
     δ₂(S)=max(0.45·0.3, 0.10·0.7)·0.2 = 0.135·0.2 = 0.0270   (desde R)
t=3: δ₃(R)=max(0.2835·0.7, 0.0270·0.3)·0.1 = 0.19845·0.1 = 0.0198 (desde R)
     δ₃(S)=max(0.2835·0.3, 0.0270·0.7)·0.8 = 0.08505·0.8 = 0.0680 (desde R)
```

Mejor camino: `q*₃ = S`, retrocediendo → **(R, R, S)** con probabilidad 0.068. Nótese que la secuencia Viterbi no es la concatenación de los estados marginalmente más probables por instante: optimiza la secuencia *completa*.

📊 Propiedades y comparación

Modelo	Estado	Observación	Inferencia	Uso típico
Cadena de Markov	visible, discreto	= estado	trivial	modelado de secuencias simples
HMM	oculto, discreto	ruidosa, discreta/continua	forward/Viterbi $O(N^2T)$	voz, POS-tagging, genes
Filtro de Kalman	oculto, continuo lineal-gaussiano	lineal + ruido gaussiano	cerrada, exacta	seguimiento, navegación
RNN/Transformer	representación aprendida	cualquiera	aproximada por gradiente	secuencias con dependencias largas

⚠️ Errores conceptuales frecuentes

1. **Confundir filtrado con decodificación.** `P(q_t|o₁…o_t)` (forward normalizado) responde "¿dónde estoy ahora?"; Viterbi responde "¿cuál fue la trayectoria completa más probable?". Pueden discrepar.
2. **Sumar los estados más probables por instante y llamarlo Viterbi.** La secuencia de máximos marginales puede incluso ser una trayectoria de probabilidad 0 (transición prohibida).
3. **Olvidar el underflow.** Con T > ~100, los productos colapsan a 0 en float64; se usa log-espacio (Viterbi) o normalización por paso (forward).
4. **Creer que Baum-Welch encuentra el óptimo global.** Es EM: óptimo local dependiente de la inicialización; se corre varias veces con semillas distintas.
5. **Aplicar Markov de orden 1 a dependencias largas sin verificarlo.** Si la observación de hoy depende de hace 10 pasos, el HMM plano lo modela mal; se amplía el estado o se cambia de familia de modelo.

🚀 Del aprendizaje a la operación

El ejemplo usa matrices dadas y 3 pasos; producción implica: estimar A y B con Baum-Welch sobre corpora grandes (con reinicios múltiples y suavizado de emisiones no vistas), trabajar íntegramente en log-espacio, elegir N (número de estados) por validación, y aceptar que los sistemas modernos de voz/lenguaje reemplazaron HMM por redes neuronales — el HMM sigue siendo el modelo de referencia cuando hay pocos datos y se necesita interpretabilidad.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Rabiner, L. R. (1989). "A tutorial on hidden Markov models and selected applications in speech recognition". *Proceedings of the IEEE*, 77(2), 257-286. https://doi.org/10.1109/5.18626 — uso: fuente primaria del mecanismo estudiado
- Viterbi, A. (1967). "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm". *IEEE Trans. Information Theory*, 13(2), 260-269. https://doi.org/10.1109/TIT.1967.1054010 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 14 "Probabilistic Reasoning over Time". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Jurafsky, D. & Martin, J. H. *Speech and Language Processing*, 3.ª ed. (draft), apéndice sobre HMM. https://web.stanford.edu/~jurafsky/slp3/ — uso: desarrollo extendido del tema
- Bishop, C. (2006). *Pattern Recognition and Machine Learning*, cap. 13. https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P03 · Memoria larga de corto plazo	1997	Primera arquitectura recurrente capaz de mantener información a través de cientos de pasos sin que el gradiente se desvanezca.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Jurafsky, Daniel y Martin, James H. — *Speech and Language Processing*	2.ª (la 3.ª circula como borrador abierto sin ISBN) · 2009	ISBN 9780131873216 · web de la obra	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 14 · obra de referencia de la parte 02
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define modelos ocultos de markov sin usar una marca o framework como definición.
2. Explica la relación entre HMM, transición, emisión, Viterbi.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

029 — Procesos de decisión de Markov

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **procesos de decisión de markov** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar procesos de decisión de markov usando los conceptos `MDP`, `estados`, `acciones`, `recompensa`, `política`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`MDP`, `estados`, `acciones`, `recompensa`, `política`

🗺️ Ubicación en el mapa de la IA

Los HMM (028) estiman el estado del mundo; los MDP dan el paso decisivo: **actuar** sobre él. Un MDP modela decisiones secuenciales bajo transiciones estocásticas, con recompensas que se acumulan en el tiempo. Formalizados por Bellman (1957), son el lenguaje matemático del aprendizaje por refuerzo (Sutton & Barto): value iteration y policy iteration son los antecesores exactos de Q-learning y de los métodos que entrenaron a AlphaGo. Junto con la utilidad esperada (030), cierran el puente entre "creer" y "decidir".

📖 Fundamentos

🧱 Definición

Un MDP es (S, A, P, R, γ):

```
S          conjunto de estados
A(s)       acciones disponibles en s
P(s'|s,a)  modelo de transición (estocástico)
R(s,a,s')  recompensa inmediata
γ ∈ [0,1)  factor de descuento
```

Propiedad de Markov: la transición depende solo del estado y acción actuales. Una **política** π: S → A prescribe qué hacer en cada estado. El objetivo: maximizar el **retorno esperado descontado** `E[Σ_t γ^t r_t]`. El descuento γ hace finita la suma infinita y codifica preferencia por recompensa temprana: una recompensa a k pasos vale γ^k veces menos.

🧮 Funciones de valor y ecuaciones de Bellman

`Vπ(s)`: retorno esperado partiendo de `s` y siguiendo π. Para la política óptima:

```
Ecuación de expectativa (política fija π):
  Vπ(s) = Σ_{s'} P(s'|s,π(s)) [ R(s,π(s),s') + γ Vπ(s') ]

Ecuación de optimalidad de Bellman:
  V*(s) = max_a Σ_{s'} P(s'|s,a) [ R(s,a,s') + γ V*(s') ]

Función Q (valor de acción):
  Q*(s,a) = Σ_{s'} P(s'|s,a) [ R(s,a,s') + γ max_{a'} Q*(s',a') ]
  π*(s) = argmax_a Q*(s,a)
```

La ecuación de optimalidad es un sistema no lineal (por el `max`) con solución única para γ < 1; se resuelve por iteración.

🔁 Value iteration

```
V₀(s) ← 0 para todo s
repetir:
    V_{k+1}(s) ← max_a Σ_{s'} P(s'|s,a) [R + γ V_k(s')]
hasta que max_s |V_{k+1}(s) − V_k(s)| < ε(1−γ)/γ
```

El operador de Bellman es una **contracción** con factor γ (teorema del punto fijo de Banach): converge geométricamente a V* desde cualquier inicialización. Costo por iteración: O(|S|²|A|).

🔁 Policy iteration

Alterna dos pasos: (1) **evaluación** — resolver `Vπ` para la política actual (sistema lineal de |S| ecuaciones, o iterativamente); (2) **mejora** — `π'(s) ← argmax_a Σ P(s'|s,a)[R + γVπ(s')]`. Si `π' = π`, es óptima. Converge en un número finito de iteraciones (hay finitas políticas y cada mejora es estricta). Suele necesitar pocas iteraciones caras, frente a muchas baratas de value iteration.

🌫️ Extensiones

- **POMDP**: el estado no se observa directamente; la política opera sobre la *creencia* (distribución sobre S, mantenida con filtrado tipo HMM). Resolverlos exactamente es intratable en general.
- **RL**: cuando P y R son desconocidos, se aprenden por interacción (Q-learning, métodos de política); el MDP sigue siendo el marco formal subyacente.

🧮 Ejemplo trabajado

MDP lineal de 4 estados $s_1–s_2–s_3–s_4$, con s_4 terminal (recompensa +10 al entrar). Acciones {→, ←}; el movimiento tiene éxito con prob. 0.8 y permanece en el sitio con 0.2. Recompensa de paso −1 por movimiento; γ = 0.9. Value iteration con $V_0 = 0$ ($V(s_4)=0$ fijo, terminal):

```
k=1:
V₁(s₃) = max→ 0.8(10 + 0.9·0) + 0.2(−1 + 0.9·0) = 8.0 − 0.2 = 7.80
V₁(s₂) = max→ 0.8(−1 + 0) + 0.2(−1 + 0) = −1.00
V₁(s₁) = −1.00

k=2:
V₂(s₃) = 0.8(10 + 0) + 0.2(−1 + 0.9·7.80) = 8.0 + 0.2·6.02 = 9.204
        (quedarse en s₃ ahora vale algo: 7.80 descontado)
V₂(s₂) = 0.8(−1 + 0.9·7.80) + 0.2(−1 + 0.9·(−1.00))
       = 0.8·6.02 + 0.2·(−1.90) = 4.816 − 0.380 = 4.436
V₂(s₁) = 0.8(−1 + 0.9·(−1.0)) + 0.2(−1 + 0.9·(−1.0)) = −1.90
```

Tras dos iteraciones ya se ve la estructura: el valor "fluye" hacia atrás desde la meta una capa por iteración, y la política `argmax` es → en todos los estados. Iterando hasta convergencia: $V^*(s_3) \approx 9.42$, $V^*(s_2) \approx 7.02$, $V^*(s_1) \approx 4.88$ (verificable sustituyendo en la ecuación de Bellman: cada valor reproduce el lado derecho).

📊 Propiedades y comparación

Método	Requiere modelo P, R	Convergencia	Costo por iteración	Cuándo usar
Value iteration	Sí	Geométrica (contracción γ)	O(	S
Policy iteration	Sí	Finita (nº de políticas)	O(	S
Q-learning (RL)	No (aprende de muestras)	Asintótica (condiciones de paso)	O(1) por transición	Modelo desconocido
POMDP exacto	Sí + modelo de observación	PSPACE-duro	exponencial	Solo problemas pequeños

⚠️ Errores conceptuales frecuentes

1. **Confundir recompensa con valor.** R es inmediata y local; V acumula el futuro descontado. Una acción con R negativa puede ser óptima si conduce a valores altos.
2. **Tratar γ como detalle técnico.** γ define el horizonte efectivo (~1/(1−γ) pasos): γ=0.5 produce agentes miopes; γ→1 puede hacer divergir la suma en problemas continuos sin estados terminales.
3. **Creer que la política óptima es determinista "por suerte".** En todo MDP con horizonte infinito descontado existe una política óptima determinista y estacionaria — es un teorema, no una casualidad.
4. **Ignorar la estocasticidad al planificar.** Elegir la acción del "mejor caso" en lugar del mejor valor *esperado* falla exactamente en los estados de riesgo (el 0.2 de fallo importa).
5. **Aplicar MDP cuando el estado no es observable.** Si el agente no sabe en qué estado está, el problema es un POMDP; usar la observación cruda como estado rompe la propiedad de Markov y las garantías.

🚀 Del aprendizaje a la operación

El laboratorio resuelve un MDP diminuto con modelo perfecto y conocido. En el mundo real: |S| suele ser astronómico (se requiere aproximación de funciones — RL profundo), P y R se desconocen o cambian (aprendizaje en línea, deriva), la recompensa mal especificada produce comportamientos indeseados (*reward hacking*), y desplegar una política implica evaluación fuera de línea, límites de seguridad y supervisión humana antes de que las decisiones toquen usuarios o dinero.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Bellman, R. (1957). *Dynamic Programming*. Princeton University Press. — uso: desarrollo extendido del tema
- Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2.ª ed., caps. 3-4. http://incompleteideas.net/book/the-book-2nd.html — uso: desarrollo extendido del tema
- Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley. https://doi.org/10.1002/9780470316887 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 17 "Making Complex Decisions". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Kaelbling, L. P., Littman, M. L. & Cassandra, A. R. (1998). "Planning and acting in partially observable stochastic domains". *Artificial Intelligence*, 101(1-2), 99-134. https://doi.org/10.1016/S0004-

3702(98)00023-X — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P26 · Control a nivel humano mediante aprendizaje por refuerzo profundo	2015	El primer agente que aprende a actuar directamente desde píxeles, con la misma arquitectura y los mismos hiperparámetros en decenas de juegos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Bellman, R. — *Dynamic Programming*	1957	ISBN 9780691079516	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 17 · obra de referencia de la parte 02
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · caps. 3-4
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define procesos de decisión de markov sin usar una marca o framework como definición.
2. Explica la relación entre MDP, estados, acciones, recompensa, política.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

030 — Teoría de decisión y utilidad esperada

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **teoría de decisión y utilidad esperada** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar teoría de decisión y utilidad esperada usando los conceptos `utilidad`, `riesgo`, `decisión`, `preferencias`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`utilidad`, `riesgo`, `decisión`, `preferencias`

🗺️ Ubicación en el mapa de la IA

La probabilidad (025-027) dice qué creer; la teoría de decisión dice qué **hacer** con esas creencias. Su núcleo — decisión racional = probabilidad + utilidad — viene de von Neumann y Morgenstern (1944) y define la noción misma de "agente racional" que organiza AIMA. Los MDP (029) son su extensión secuencial; las redes de decisión extienden las redes bayesianas (027) con nodos de acción y utilidad; y la crítica empírica (Kahneman & Tversky) delimita cuándo los humanos NO deciden así.

📖 Fundamentos

🎯 El principio de máxima utilidad esperada (MEU)

Dada una acción `a` con resultados inciertos y una función de utilidad `U`:

```
EU(a | e) = Σ_{s'} P(Resultado = s' | a, e) · U(s')

acción racional:  a* = argmax_a EU(a | e)
```

La utilidad no es dinero: es una escala numérica de **preferencias**. El dinero suele tener utilidad marginal decreciente (el primer millón cambia la vida; el décimo, no).

📜 Axiomas de von Neumann–Morgenstern

Sobre **loterías** $[p_1, r_1; \ldots; p_n, r_n]$ (resultados con probabilidades), un agente con preferencias que cumplen:

1. **Completitud**: $A \succ B$, $B \succ A$, o $A \sim B$.
2. **Transitividad**: $A \succ B$ y $B \succ C \Rightarrow A \succ C$.
3. **Continuidad**: si $A \succ B \succ C$, existe p con $B \sim [p, A; 1{-}p, C]$.
4. **Independencia**: $A \succ B \Rightarrow [p, A; 1{-}p, C] \succ [p, B; 1{-}p, C]$.

…se comporta **como si** maximizara la esperanza de alguna función de utilidad U, única salvo transformación afín positiva ($aU + b$, $a > 0$). Violar los axiomas expone al agente a ser explotado como "bomba de dinero" (money pump): ciclos de intercambios donde pierde en cada vuelta.

⚖️ Actitud frente al riesgo

Para utilidad sobre dinero U(x):

```
U cóncava  → aversión al riesgo:    U(E[X]) > E[U(X)]  (prefiere lo seguro)
U lineal   → neutralidad al riesgo: decide por valor esperado
U convexa  → propensión al riesgo
```

- **Equivalente cierto (CE)**: cantidad segura tal que `U(CE) = E[U(X)]`.
- **Prima de riesgo**: `E[X] − CE` — cuánto "paga" el agente por eliminar la incertidumbre; es la base económica de los seguros.

💡 Valor de la información (VPI)

Cuánto vale observar una variable E antes de decidir:

```
VPI(E) = [ Σ_e P(e) · max_a EU(a | e) ] − max_a EU(a)
```

Siempre `VPI ≥ 0` (la información no obliga a cambiar de decisión, solo lo permite) y vale 0 si ninguna observación cambiaría la acción elegida. Guía racionalmente qué test pedir, qué sensor instalar, qué pregunta hacer.

🧭 Redes de decisión

Una red bayesiana + **nodos de decisión** (rectángulos) + **nodo de utilidad** (rombo). La evaluación: para cada valor de la decisión, propagar probabilidades y promediar utilidades; elegir el máximo. Es el formalismo gráfico del MEU.

🧠 La crítica empírica

Allais (1953) y Ellsberg mostraron violaciones sistemáticas del axioma de independencia; Kahneman & Tversky (1979) las organizaron en la **teoría prospectiva**: los humanos evalúan cambios respecto a un punto de referencia, sienten las pérdidas ~2× más que las ganancias y distorsionan probabilidades pequeñas.

Consecuencia para IA: los modelos de usuario no deben asumir MEU, pero el *agente artificial* sí puede diseñarse para cumplirlo.

🧮 Ejemplo trabajado

¿Contratar un seguro? Riqueza inicial w = 10 000 €. Riesgo: perder 5 000 € con probabilidad 0.1. Prima del seguro: 600 €. Utilidad logarítmica `U(x) = ln(x)` (aversión al riesgo).

```
Sin seguro:
EU = 0.9·ln(10000) + 0.1·ln(5000)
   = 0.9·9.2103 + 0.1·8.5172
   = 8.2893 + 0.8517 = 9.1410

Con seguro (riqueza segura 9400):
EU = ln(9400) = 9.1485

9.1485 > 9.1410  →  contratar el seguro es racional…
…aunque su valor esperado monetario sea peor:
  sin seguro: E[X] = 0.9·10000 + 0.1·5000 = 9500 > 9400
```

Equivalente cierto sin seguro: `CE = e^9.1410 ≈ 9330 €`. El agente es indiferente entre el riesgo y 9 330 € seguros; como el seguro le deja 9 400 € > CE, lo compra. **Prima de riesgo** = 9 500 − 9 330 = 170 €: la aseguradora puede cobrar hasta 670 € sobre la pérdida esperada (500 €) y ambos salen ganando en utilidad.

VPI: si un peritaje perfecto (gratis) revelara si ocurrirá el siniestro, la decisión sería: asegurar solo si ocurrirá. `EU_info = 0.9·ln(10000) + 0.1·ln(9400·? )` — con seguro "a posteriori" el cálculo muestra que la información perfecta vale más que la prima en escenarios de riesgo alto; con ella, nadie compraría seguros: por eso las aseguradoras temen la selección adversa.

📊 Propiedades y comparación

Criterio de decisión	Usa probabilidades	Usa preferencias graduadas	Riesgo	Falla típica
Máximo valor esperado (dinero)	Sí	No (lineal)	Neutral	Ignora ruina: apuesta de San Petersburgo
MEU (utilidad esperada)	Sí	Sí	Configurable	Exige elicitar U y P
Maximin (peor caso)	No	Ordinal	Extremadamente averso	Paraliza ante riesgos minúsculos
Minimax regret	No	Diferencias	Intermedio	Sensible a alternativas irrelevantes
Humanos reales (prospectiva)	Distorsionadas	Ref.-dependientes	Asimétrico pérdidas/ganancias	Incoherencia explotable

⚠️ Errores conceptuales frecuentes

1. **Utilidad = dinero.** Maximizar valor esperado monetario justificaría rechazar todo seguro y aceptar la apuesta de San Petersburgo; la concavidad de U explica el comportamiento razonable.
2. **"La utilidad es única."** Solo lo es salvo transformación afín: U y 3U+7 producen exactamente las mismas decisiones; comparar utilidades absolutas entre agentes carece de sentido.
3. **Decidir por la hipótesis más probable.** Sin costos, el argmax de probabilidad puede ser pésimo: un 2 % de incendio domina la decisión si el costo del incendio es enorme.
4. **VPI negativo o "la información puede hacer daño".** Formalmente VPI ≥ 0 para un agente racional; lo que sí puede ser negativo es el valor *neto* (información menos su costo).
5. **Asumir que los usuarios humanos maximizan utilidad esperada.** La evidencia (Allais, framing, aversión a pérdidas) dice lo contrario; los sistemas que modelan personas necesitan modelos descriptivos, no solo normativos.

🚀 Del aprendizaje a la operación

El ejemplo asume U y P conocidas y estables. Operacionalmente: elicitar utilidades es difícil y ruidoso (se hace con loterías de referencia o preferencias reveladas), las probabilidades vienen de modelos con error de calibración, los objetivos múltiples exigen utilidades multiatributo con trade-offs explícitos, y una función de utilidad mal especificada en un agente autónomo produce optimización de lo que se midió, no de lo que se quería — el problema de alineación en miniatura.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;

- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- von Neumann, J. & Morgenstern, O. (1944). *Theory of Games and Economic Behavior*. Princeton University Press. — uso: desarrollo extendido del tema
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 16 "Making Simple Decisions". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Kahneman, D. & Tversky, A. (1979). "Prospect Theory: An Analysis of Decision under Risk". *Econometrica*, 47(2), 263-291. https://doi.org/10.2307/1914185 — uso: fuente primaria del mecanismo estudiado
- Howard, R. A. (1966). "Information Value Theory". *IEEE Trans. Systems Science and Cybernetics*, 2(1), 22-26. https://doi.org/10.1109/TSSC.1966.300074 — uso: fuente primaria del mecanismo estudiado
- Savage, L. J. (1954). *The Foundations of Statistics*. Wiley. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P26 · Control a nivel humano mediante aprendizaje por refuerzo profundo	2015	El primer agente que aprende a actuar directamente desde píxeles, con la misma arquitectura y los mismos hiperparámetros en decenas de juegos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 16 · obra de referencia de la parte 02
Savage, L. J. — *The Foundations of Statistics*	1954	ISBN 9780486623498	citada en las referencias de esta clase
von Neumann, J. y Morgenstern, O. — *Theory of Games and Economic Behavior*	1944	ISBN 9781777257316	citada en las referencias de esta clase
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define teoría de decisión y utilidad esperada sin usar una marca o framework como definición.
2. Explica la relación entre utilidad, riesgo, decisión, preferencias.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

031 — Métodos Monte Carlo y simulación

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **métodos monte carlo y simulación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar métodos monte carlo y simulación usando los conceptos `Monte Carlo`, `muestreo`, `estimación`, `varianza`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`Monte Carlo`, `muestreo`, `estimación`, `varianza`

🗺️ Ubicación en el mapa de la IA

Las clases 025-030 calculan probabilidades y utilidades de forma exacta; eso deja de ser viable cuando el modelo es grande (inferencia NP-dura en redes bayesianas) o la integral no tiene forma cerrada. Monte Carlo — nacido en Los Álamos con Ulam, von Neumann y Metropolis (1949) — sustituye el cálculo por **muestreo**: estimar cantidades como promedios de simulaciones aleatorias. Sostiene la inferencia aproximada en modelos gráficos, la programación probabilística (035), el MCTS de AlphaGo y buena parte del RL moderno.

📖 Fundamentos

🎯 El estimador Monte Carlo

Para estimar `μ = E[f(X)]` con `X ~ p`:

```
μ̂_N = (1/N) Σ_{i=1}^{N} f(x_i),   x_i ~ p  i.i.d.

Ley de los grandes números:  μ̂_N → μ  (consistencia)
TCL: error estándar  σ/√N,  con σ² = Var[f(X)]
```

La propiedad clave: la tasa de convergencia `O(1/√N)` **no depende de la dimensión** del problema — de ahí su ventaja sobre cuadraturas deterministas en alta dimensión. La contrapartida: cada dígito decimal extra de precisión cuesta ×100 muestras.

🎲 Muestreo en modelos probabilísticos

- **Muestreo directo (prior sampling)**: en una red bayesiana, muestrear cada nodo en orden topológico según su CPT. Genera mundos con la frecuencia de la conjunta.
- **Muestreo con rechazo**: para `P(Q|e)`, descartar las muestras incompatibles con `e`. Correcto pero desperdicia una fracción `1 − P(e)` de muestras: inútil si la evidencia es improbable.
- **Ponderación por verosimilitud (likelihood weighting)**: fijar las variables de evidencia y ponderar cada muestra por $\prod P(e_i|\text{padres})$. Nada se descarta, pero con mucha evidencia los pesos degeneran (pocas muestras dominan).
- **MCMC (Metropolis-Hastings, Gibbs)**: construir una cadena de Markov cuya distribución estacionaria es la posterior; tras un periodo de calentamiento (*burn-in*), los estados visitados son muestras (correlacionadas) de la posterior. Escala a problemas donde lo anterior falla.

📉 Reducción de varianza

La precisión depende de σ^2, no solo de N:

- **Muestreo por importancia**: muestrear de `q` y ponderar por `p/q`; una `q` bien elegida concentra muestras donde `|f|·p` es grande.
- **Variables antitéticas**, **variables de control**, **estratificación**: correlacionar o descomponer para cancelar ruido.

🌲 De la estimación a la decisión: MCTS

Monte Carlo Tree Search estima el valor de acciones simulando partidas completas (*rollouts*) y equilibra exploración/explotación con UCB. Es el puente entre esta clase y los MDP (029): cuando el árbol es demasiado grande para Bellman exacto, se muestrea.

🔁 Reproducibilidad

Los generadores son **pseudo**aleatorios: una semilla fija produce la misma secuencia. En ciencia e ingeniería la semilla es parte del experimento: sin ella, un resultado Monte Carlo no es auditable. Reportar siempre: semilla, N, estimador y error estándar.

🧮 Ejemplo trabajado

Estimar π lanzando dardos. Se muestrean puntos uniformes en el cuadrado $[0,1]^2$; la fracción que cae dentro del cuarto de círculo ($x^2+y^2 \le 1$) estima $\pi/4$.

```
π̂ = 4 · (aciertos / N)

f = 1_{x²+y²≤1} es Bernoulli(p = π/4 ≈ 0.7854)
Var[f] = p(1−p) ≈ 0.1685  →  σ ≈ 0.4105
Error estándar de π̂: 4·σ/√N ≈ 1.642/√N
```

N	error estándar de $\hat{\pi}$	resultado típico
100	0.164	3.0 – 3.3
10 000	0.0164	3.12 – 3.17
1 000 000	0.00164	3.138 – 3.145

Cálculo a mano con N = 20 (semilla fija, 16 aciertos): `π̂ = 4·16/20 = 3.2`; el intervalo ±1 e.e. es `3.2 ± 1.64/√20 ≈ 3.2 ± 0.37` — consistente con π. Duplicar la precisión exige cuadruplicar N: de ahí que Monte Carlo se combine con reducción de varianza en lugar de fuerza bruta.

Ponderación por verosimilitud en la red de la alarma (027): para `P(B | j, m)` se fijan J = j y M = m; una muestra `(b=no, e=no, a=no)` recibe peso `P(j|¬a)·P(m|¬a) = 0.05·0.01 = 0.0005`, mientras una con `a=sí` recibe `0.9·0.7 = 0.63`: las muestras compatibles con la evidencia dominan la estimación sin descartar ninguna.

📊 Propiedades y comparación

Método	Sesgo	Eficiencia con evidencia rara	Muestras correlacionadas	Cuándo usar
Cuadratura determinista	0	N/A	N/A	Dimensión baja (≤ 3-4)
Muestreo directo	No	No aplica (sin evidencia)	No	Simular el modelo a priori
Rechazo	No	Pésima: descarta 1−P(e)	No	Evidencia probable
Likelihood weighting	No (ponderado)	Media: pesos degeneran	No	Evidencia moderada
MCMC	Asintóticamente no	Buena	Sí (autocorrelación)	Posteriores complejas, alta dimensión

⚠️ Errores conceptuales frecuentes

1. **Reportar la estimación sin el error.** `π̂ = 3.2` no dice nada sin `± 0.37`; el estimador es una variable aleatoria y su incertidumbre es parte del resultado.
2. **"Con más muestras siempre basta."** La convergencia 1/√N es lenta: pasar de 2 a 3 decimales cuesta 100× más cómputo; la reducción de varianza suele rendir más que aumentar N.

3. **Confundir pseudoaleatorio con aleatorio.** La semilla fija hace reproducible el experimento; cambiarla y ver si la conclusión se sostiene es parte de la validación, no un detalle.
4. **Usar muestras MCMC como si fueran i.i.d.** Están autocorrelacionadas: el "tamaño efectivo de muestra" puede ser 10× menor que N, y olvidar el burn-in sesga la estimación.
5. **Muestreo con rechazo ante evidencia improbable.** Con `P(e) = 10⁻⁴`, el 99.99 % del cómputo se tira; hay que cambiar de algoritmo, no de paciencia.

🚀 Del aprendizaje a la operación

Estimar π es un juguete con respuesta conocida; en producción la respuesta no se conoce y la validación es indirecta: diagnósticos de convergencia MCMC ($\hat{R}$ de Gelman-Rubin, tamaño efectivo), comparación entre semillas y muestreadores, pruebas con casos límite calculables y presupuesto de cómputo explícito. Además, las simulaciones heredan los errores del modelo: Monte Carlo cuantifica la incertidumbre *dentro* del modelo, nunca la de haber elegido el modelo equivocado.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Metropolis, N. & Ulam, S. (1949). "The Monte Carlo Method". *Journal of the American Statistical Association*, 44(247), 335-341. https://doi.org/10.1080/01621459.1949.10483310 — uso: fuente primaria del mecanismo estudiado
- Metropolis, N. et al. (1953). "Equation of State Calculations by Fast Computing Machines". *J. Chemical Physics*, 21(6), 1087-1092. https://doi.org/10.1063/1.1699114 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 13.4 (inferencia aproximada). https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Robert, C. & Casella, G. (2004). *Monte Carlo Statistical Methods*, 2.ª ed. Springer. https://doi.org/10.1007/978-1-4757-4145-2 — uso: fuente primaria del mecanismo estudiado
- Sutton, R. S. & Barto, A. G. (2018). *Reinforcement Learning*, cap. 5 (métodos Monte Carlo). http://incompleteideas.net/book/the-book-2nd.html — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P27 · Dominar el go con redes neuronales profundas y búsqueda en árbol	2016	Une las dos tradiciones de la IA: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04, en un solo sistema.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 13 · obra de referencia de la parte 02
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · cap. 5
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define métodos monte carlo y simulación sin usar una marca o framework como definición.
2. Explica la relación entre Monte Carlo, muestreo, estimación, varianza.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

032 — Lógica difusa y control aproximado

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **lógica difusa y control aproximado** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lógica difusa y control aproximado usando los conceptos `fuzzy`, `membresía`, `reglas`, `control`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`fuzzy`, `membresía`, `reglas`, `control`

🗺️ Ubicación en el mapa de la IA

La probabilidad (025-031) modela **ignorancia**: no sé si lloverá. La lógica difusa (Zadeh, 1965) modela algo distinto, **vaguedad**: "hace calor" no es verdadero ni falso, es cuestión de grado. Nació como crítica a la rigidez de los conjuntos clásicos y triunfó en el control industrial (metro de Sendai, cámaras, lavadoras) porque traduce reglas lingüísticas de expertos a controladores continuos sin ecuaciones diferenciales. En el mapa de la IA es la rama "aproximada" del razonamiento simbólico, complementaria — no rival — de la probabilística.

📖 Fundamentos

🌡️ Conjuntos difusos y funciones de membresía

Un conjunto clásico tiene función característica `χ_A: X → {0,1}`; un **conjunto difuso** la generaliza a una **función de membresía** `μ_A: X → [0,1]`. `μ_calor(28 °C) = 0.7` significa que 28 °C pertenece "al 70 %" al concepto *calor*. Formas típicas: triangular, trapezoidal, gaussiana. Una **variable lingüística** (Temperatura) toma valores lingüísticos (fría, templada, caliente), cada uno un conjunto difuso sobre el mismo universo.

🔗 Operadores

```
Negación:      μ_¬A(x) = 1 − μ_A(x)
Intersección:  μ_{A∧B}(x) = min(μ_A, μ_B)     (t-norma; también producto)
Unión:         μ_{A∨B}(x) = max(μ_A, μ_B)     (t-conorma)
```

Consecuencia notable: en general `μ_{A∧¬A} ≠ 0` — se pierde el principio de no contradicción. No es un defecto: 24 °C puede ser "algo cálido y algo fresco" a la vez. Esto marca la diferencia semántica con la probabilidad, donde `P(A ∧ ¬A) = 0` siempre.

⚙️ Sistema de inferencia difusa (Mamdani)

Un controlador difuso ejecuta cuatro etapas:

```
1. Fuzzificación:  entrada nítida x → grados μ de cada conjunto de entrada
2. Evaluación de reglas: "SI temp es alta Y humedad es baja
                          ENTONCES ventilador es rápido"
   fuerza de disparo w = min(μ_alta(temp), μ_baja(hum))
3. Agregación: cada consecuente se recorta a su w (implicación min);
   los consecuentes de todas las reglas se unen con max
4. Defuzzificación: el conjunto agregado → un número nítido
   (centroide: y* = ∫ y·μ(y) dy / ∫ μ(y) dy)
```

La variante **Sugeno** usa consecuentes funcionales (`y = f(x)`) y promedio ponderado en lugar de centroide: más eficiente y apta para optimización automática (ANFIS), menos interpretable lingüísticamente.

🧭 Cuándo tiene sentido

La lógica difusa brilla cuando: (a) existe experiencia humana expresable en reglas lingüísticas, (b) el sistema tolera control subóptimo pero suave, (c) modelar la planta con ecuaciones es caro. No aprende de datos por sí sola ni cuantifica incertidumbre epistémica: para eso están los modelos probabilísticos.

🧮 Ejemplo trabajado

Controlador de ventilador con una entrada (Temperatura, universo 0-40 °C) y una salida (Velocidad, 0-100 %).

Membresías triangulares de entrada: `fría` = triángulo(0, 0, 20); `templada` = triángulo(10, 20, 30); `caliente` = triángulo(20, 40, 40). Salida: `lenta` = tri(0, 0, 50); `media` = tri(25, 50, 75); `rápida` = tri(50, 100, 100).

Reglas: R1: fría → lenta; R2: templada → media; R3: caliente → rápida.

Entrada nítida: T = 26 °C.

```
Fuzzificación:
μ_fría(26)     = 0                       (26 > 20)
μ_templada(26) = (30−26)/(30−20) = 0.4   (rama descendente)
μ_caliente(26) = (26−20)/(40−20) = 0.3   (rama ascendente)

Disparo: R2 con w=0.4 (media recortada a 0.4)
         R3 con w=0.3 (rápida recortada a 0.3)

Defuzzificación (centroide aproximado por muestreo cada 12.5):
y:      25    37.5   50    62.5   75    87.5   100
μ_med:  0     0.4    0.4   0.4    0     0      0     (recortada a 0.4)
μ_ráp:  0     0      0     0.25   0.3   0.3    0.3   (recortada a 0.3)
μ_agg:  0     0.4    0.4   0.4    0.3   0.3    0.3   (máximo)

y* = Σ y·μ / Σ μ
   = (37.5·0.4 + 50·0.4 + 62.5·0.4 + 75·0.3 + 87.5·0.3 + 100·0.3) / (0.4·3 + 0.3·3)
   = (15 + 20 + 25 + 22.5 + 26.25 + 30) / 2.1
   = 138.75 / 2.1 ≈ 66.1 %
```

El ventilador gira al ~66 %: ni "media" ni "rápida", sino una mezcla ponderada por cuán templado y cuán caliente está. Si T sube a 27 °C, la salida sube suavemente (≈ 68 %) — sin el salto brusco de un termostato con umbral.

📊 Propiedades y comparación

Aspecto	Lógica difusa	Probabilidad	Control clásico (PID)
Qué modela	Vaguedad de predicados	Ignorancia sobre hechos	Dinámica de la planta
μ(x)=0.7 significa	Pertenencia parcial	70 % de creencia en hecho nítido	N/A
A ∧ ¬A	Puede ser > 0	Siempre 0	N/A
Fuente del modelo	Reglas de experto	Datos / axiomas	Ecuaciones / ajuste
Aprendizaje	No nativo (ANFIS lo añade)	Estimación estadística	Sintonización
Garantías de estabilidad	Difíciles de probar	N/A	Teoría madura

⚠️ Errores conceptuales frecuentes

1. **"Difuso = probabilístico."** `μ_calor(28) = 0.7` no es `P(calor) = 0.7`: no hay experimento aleatorio; 28 °C es perfectamente conocido, lo vago es el predicado. Vaguedad ≠ ignorancia.

2. **Creer que las membresías son objetivas.** Son elecciones de diseño (¿dónde empieza "caliente"?); dos ingenieros producen controladores distintos y ambos "correctos". Se validan por desempeño, no por verdad.
3. **Esperar que el sistema aprenda.** Un FIS puro no ajusta nada con datos; sin ANFIS u optimización externa, las reglas erróneas permanecen erróneas.
4. **Ignorar la explosión de reglas.** Con k entradas y m etiquetas cada una, la tabla completa tiene m^k reglas; 5 entradas con 7 etiquetas → 16 807 reglas: inviable de elicitar y mantener.
5. **Usar difusa donde hay incertidumbre estadística real.** Para diagnóstico con tasas de error medibles, Bayes (026) da garantías que min/max no puede dar.

🚀 Del aprendizaje a la operación

El ejemplo tiene 1 entrada y 3 reglas; un controlador real tiene varias entradas, decenas de reglas y requisitos de estabilidad. Falta aquí: sintonización de membresías con datos (ANFIS/optimización, clases 033-034 sirven para esto), análisis de estabilidad y de casos extremos del actuador, latencia y saturación del hardware, y comparación honesta contra un PID bien sintonizado — que en muchas plantas simples gana con menos complejidad.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Zadeh, L. A. (1965). "Fuzzy Sets". *Information and Control*, 8(3), 338-353. https://doi.org/10.1016/S0019-9958(65)90241-X — uso: fuente primaria del mecanismo estudiado
- Mamdani, E. H. & Assilian, S. (1975). "An experiment in linguistic synthesis with a fuzzy logic controller". *Int. J. Man-Machine Studies*, 7(1), 1-13. https://doi.org/10.1016/S0020-7373(75)80002-2 — uso: fuente primaria del mecanismo estudiado
- Takagi, T. & Sugeno, M. (1985). "Fuzzy identification of systems and its applications to modeling and control". *IEEE Trans. SMC*, 15(1), 116-132. https://doi.org/10.1109/TSMC.1985.6313399 — uso: fuente primaria del mecanismo estudiado
- Jang, J.-S. R. (1993). "ANFIS: adaptive-network-based fuzzy inference system". *IEEE Trans. SMC*, 23(3), 665-685. https://doi.org/10.1109/21.256541 — uso: fuente primaria del mecanismo estudiado

- Ross, T. J. (2010). *Fuzzy Logic with Engineering Applications*, 3.ª ed. Wiley. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P89 · Conjuntos difusos	1965	Permite que un elemento pertenezca parcialmente a un conjunto, y con eso da tratamiento formal a la vaguedad de los predicados del lenguaje.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Ross, T. J. — *Fuzzy Logic with Engineering Applications*	2010	ISBN 9780071136372	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 02 · bloque de incertidumbre y decisión
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define lógica difusa y control aproximado sin usar una marca o framework como definición.

2. Explica la relación entre fuzzy, membresía, reglas, control.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

033 — Algoritmos genéticos

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `optimization` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **algoritmos genéticos** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar algoritmos genéticos usando los conceptos `población`, `mutación`, `cruce`, `selección`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`población`, `mutación`, `cruce`, `selección`

🗺️ Ubicación en el mapa de la IA

Hasta ahora la parte 02 razona y decide con modelos explícitos; los algoritmos genéticos (Holland, 1975; Goldberg, 1989) atacan otro problema: **optimizar** funciones sin gradiente, sin convexidad y sin forma cerrada, imitando la evolución darwiniana. Pertenecen a la familia de metaheurísticas poblacionales junto con PSO y ACO (034). En la práctica moderna optimizan hiperparámetros, arquitecturas neuronales (neuroevolución), planificación y diseño — y son el recordatorio histórico de que la búsqueda estocástica compite con el cálculo cuando el paisaje es hostil.

📖 Fundamentos

🧬 Metáfora y componentes

- **Individuo (cromosoma)**: candidato a solución, codificado (bits, enteros, reales, permutaciones).
- **Población**: multiconjunto de N individuos; la diversidad es su capital.
- **Fitness** f(x): función a maximizar; única señal que guía la búsqueda.
- **Selección**: los más aptos se reproducen con mayor probabilidad — explota.
- **Cruce (crossover)**: combina dos padres — recombina bloques buenos.

- **Mutación**: perturbación aleatoria pequeña — explora y evita estancamiento.
- **Elitismo**: copiar los k mejores intactos garantiza no perder lo logrado.

⚙️ Bucle canónico

```
P ← población inicial aleatoria (tamaño N), evaluar fitness
repetir hasta criterio de parada:
    E ← mejores k de P                      # elitismo
    repetir hasta llenar la nueva población:
        p₁, p₂ ← seleccionar(P)             # torneo o ruleta
        h₁, h₂ ← cruzar(p₁, p₂) con prob. pc # pc ≈ 0.6–0.9
        mutar(h₁), mutar(h₂) con prob. pm    # pm ≈ 1/longitud
    P ← E ∪ hijos; evaluar fitness
devolver el mejor individuo visto
```

Selección por torneo (tamaño t): elegir t individuos al azar y quedarse con el mejor; t controla la **presión selectiva** (t grande → convergencia rápida y pérdida de diversidad). **Ruleta**: probabilidad proporcional al fitness; sensible a la escala (un superindividuo temprano puede monopolizar).

Cruce de un punto en cadenas: cortar ambos padres en una posición y recombinar. Para **permutaciones** (TSP) el cruce debe preservar la validez (OX, PMX); para **reales** se usa cruce aritmético o SBX y mutación gaussiana.

📈 Por qué funciona (y cuándo no)

La intuición clásica es el **teorema de esquemas** de Holland: los patrones cortos, de bajo orden y fitness sobre el promedio ("bloques constructivos") reciben exponencialmente más copias en generaciones sucesivas. Es una intuición útil más que una garantía: el teorema tiene supuestos fuertes y no asegura convergencia al óptimo global. El **teorema No Free Lunch** (Wolpert & Macready, 1997) añade la advertencia definitiva: promediado sobre todos los problemas posibles, ningún optimizador supera a otro — un GA solo gana donde la estructura del problema casa con sus operadores.

🎛️ El equilibrio exploración/explotación

Todo el diseño (N, pc, pm, presión selectiva, elitismo) regula un único trade-off: explotar las buenas regiones halladas vs. seguir explorando. Convergencia prematura (población clonada en un óptimo local) es el modo de fallo dominante; sus antídotos: mutación suficiente, torneos pequeños, nichos o reinicios.

🧮 Ejemplo trabajado

OneMax: maximizar el número de unos en una cadena de 8 bits (óptimo = 8). Población N = 4, torneo de 2, cruce de un punto (punto 4), mutación de un bit, elitismo 1.

```
Generación 0                    fitness
A = 1 0 1 1 0 1 0 0               4
B = 0 1 1 0 1 0 1 0               4
C = 1 1 0 1 0 0 0 1               4
D = 0 0 0 1 0 1 0 0               2

Torneos: (A,D)→A  (B,C)→B  (C,D)→C  (A,B)→A (empate: primero)

Cruce A×B por el punto 4:
  A = 1011|0100, B = 0110|1010
  h₁ = 1011 1010 (f=5)   h₂ = 0110 0100 (f=3)
Cruce C×A por el punto 4:
  h₃ = 1101 0100 (f=4)   h₄ = 1011 0001 (f=4)

Mutación (1 bit al azar): h₂: bit 6 0→1 ⇒ 0110 0110 (f=4)

Generación 1 = {elite A(4), h₁(5), h₂(4), h₃(4)}
mejor: h₁ = 10111010 con f = 5  (mejora sobre 4)
```

Dos observaciones didácticas: (1) el cruce creó h_1 con 5 unos combinando la mitad izquierda rica de A con la derecha rica de B — el "bloque constructivo" en acción; (2) D, el peor, no sobrevivió a ningún torneo: presión selectiva. Repitiendo el proceso, OneMax converge al óptimo en pocas decenas de generaciones; con `pm = 0` la población puede clonarse antes de llegar (convergencia prematura demostrable con este mismo ejemplo).

📊 Propiedades y comparación

Método	Requiere gradiente	Población	Garantía de óptimo	Coste por iteración	Nicho de uso
Descenso de gradiente	Sí	No	Local (convexo: global)	Bajo	f diferenciable
Hill climbing + reinicios	No	No	Local	Bajo	Paisajes suaves
Recocido simulado	No	No	Global (enfriamiento ∞-lento)	Bajo	Combinatoria mediana
Algoritmo genético	No	Sí	Ninguna práctica	Alto (N evaluaciones)	Paisajes rugosos, codificación natural
CMA-ES	No	Sí	Ninguna, fuerte en continuo	Medio	Optimización continua ≤ ~100 dim

⚠️ Errores conceptuales frecuentes

1. **"El GA encuentra el óptimo global."** No hay garantía práctica; devuelve el mejor visto. Se corre varias veces con semillas distintas y se reporta la distribución, no una corrida.
2. **Confundir fitness alto en la población con progreso.** Si toda la población es idéntica, el fitness medio es alto pero la búsqueda murió (convergencia prematura); hay que monitorear diversidad.
3. **Mutación como protagonista.** Con pm alto el GA degenera en búsqueda aleatoria; la mutación es un seguro contra pérdida de alelos, el cruce hace el trabajo de recombinación.
4. **Codificación descuidada.** Un cruce de un punto sobre una permutación produce hijos inválidos (ciudades repetidas en TSP); el operador debe respetar la semántica de la representación.
5. **Comparar contra nada.** Sin baseline (búsqueda aleatoria, hill climbing con reinicios) no se puede afirmar que el GA "funciona"; a menudo el baseline gana en problemas fáciles con presupuesto igual de evaluaciones.

🚀 Del aprendizaje a la operación

OneMax se evalúa en microsegundos; en problemas reales la evaluación del fitness domina el costo (simulaciones, entrenamientos), lo que impone paralelización, fitness aproximado (surrogates) y presupuestos estrictos de evaluaciones. Faltan además: sintonización de hiperparámetros del propio GA (meta-optimización), manejo de restricciones (penalización o reparación), y protocolo estadístico de comparación (múltiples semillas, tests no paramétricos) antes de reclamar superioridad sobre alternativas.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("optimization")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press (reed. MIT Press, 1992). — uso: desarrollo extendido del tema
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley. — uso: desarrollo extendido del tema
- Wolpert, D. H. & Macready, W. G. (1997). "No Free Lunch Theorems for Optimization". *IEEE Trans. Evolutionary Computation*, 1(1), 67-82. https://doi.org/10.1109/4235.585893 — uso: fuente primaria del mecanismo estudiado
- Eiben, A. E. & Smith, J. E. (2015). *Introduction to Evolutionary Computing*, 2.ª ed. Springer. https://doi.org/10.1007/978-3-662-44874-8 — uso: fuente primaria del mecanismo estudiado

- Russell, S. & Norvig, P. (2020). *AIMA*, 4.ª ed., cap. 4.1 (búsqueda local y evolutiva). https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P90 · Algoritmos genéticos y la asignación óptima de ensayos	1973	Conecta la evolución artificial con un problema de decisión clásico: cómo repartir ensayos entre alternativas cuando explorar cuesta.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Holland, J. H. — *Adaptation in Natural and Artificial Systems*	1975	ISBN 9780262082136	citada en las referencias de esta clase
Goldberg, D. E. — *Genetic Algorithms in Search, Optimization, and Machine Learning*	1989	ISBN 9780201157673	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 4 · obra de referencia de la parte 02
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define algoritmos genéticos sin usar una marca o framework como definición.
2. Explica la relación entre población, mutación, cruce, selección.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

034 — Optimización por enjambre y colonia

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `optimization` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **optimización por enjambre y colonia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar optimización por enjambre y colonia usando los conceptos `PSO`, `ACO`, `metaheurística`, `exploración`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`PSO`, `ACO`, `metaheurística`, `exploración`

🗺️ Ubicación en el mapa de la IA

Los GA (033) evolucionan una población por selección y recombinación; la **inteligencia de enjambre** logra optimización con un mecanismo distinto: agentes simples que cooperan mediante interacciones locales, sin selección ni cromosomas. PSO (Kennedy & Eberhart, 1995) imita bandadas que comparten la mejor posición conocida; ACO (Dorigo, años 90) imita hormigas que depositan feromona sobre buenas rutas. Cierran el bloque bio-inspirado del curso y anticipan una idea mayor: comportamiento colectivo inteligente **emergente** de reglas locales — el mismo principio detrás de los sistemas multiagente.

📖 Fundamentos

🐓 PSO — Particle Swarm Optimization (optimización continua)

Cada partícula i tiene posición x_i, velocidad v_i, su mejor posición histórica p_i (memoria personal) y conoce la mejor global g (memoria social). Actualización por dimensión:

```
v_i ← w·v_i + c₁·r₁·(p_i − x_i) + c₂·r₂·(g − x_i)
x_i ← x_i + v_i

w  ≈ 0.4–0.9   inercia (exploración)
c₁ ≈ 2         componente cognitiva (volver a lo mío)
c₂ ≈ 2         componente social (ir hacia lo del grupo)
r₁, r₂ ~ U(0,1) por dimensión
```

Tres fuerzas en tensión: inercia (seguir explorando), nostalgia (mi mejor punto), conformismo (el mejor del enjambre). Con `w` alto el enjambre explora; al reducir `w` con el tiempo, converge. Variantes: topología de vecindarios (lbest vs. gbest — gbest converge rápido pero se atasca más), factor de constricción de Clerc para garantizar estabilidad.

🐜 ACO — Ant Colony Optimization (optimización combinatoria)

Para problemas de rutas (TSP): m hormigas construyen soluciones paso a paso, eligiendo la arista (i,j) con probabilidad que mezcla **feromona** τ (memoria colectiva) y **heurística** η = 1/distancia:

$$P(i \to j) = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\Sigma_k \, \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}} \qquad (j \text{ no visitada})$$

$$\text{Actualización de feromona tras cada iteración:}$$

$$\tau_{ij} \leftarrow (1-\rho)\cdot\tau_{ij} + \Sigma_\text{hormigas}\ \Delta\tau_{ij}\ , \quad \Delta\tau_{ij} = Q/L \text{ si la hormiga usó } (i,j)$$

- α pondera la feromona (experiencia colectiva), β la heurística (miopía codiciosa).
- ρ ∈ (0,1] es la **evaporación**: olvida rutas viejas y evita la congelación temprana.
- El refuerzo Q/L premia rutas cortas: retroalimentación positiva — las buenas aristas reciben más feromona, atraen más hormigas, reciben aún más feromona.

La **estigmergia** (comunicación a través del entorno, no entre agentes) es el concepto central: ninguna hormiga conoce el mapa; el conocimiento vive en la matriz de feromonas.

🔁 El patrón común

Ambos algoritmos instancian el mismo esquema: (1) población de constructores estocásticos, (2) memoria compartida de calidad (g en PSO, τ en ACO), (3) retroalimentación positiva moderada por un mecanismo de olvido (inercia decreciente, evaporación), (4) equilibrio exploración/explotación gobernado por 2-3 hiperparámetros. Comparten también el modo de fallo: **estancamiento** cuando la memoria colectiva se vuelve dogma.

🧮 Ejemplo trabajado

PSO a mano en 1D: minimizar `f(x) = x²`. Dos partículas, w = 0.5, c1 = c2 = 1, y para seguirlo a mano fijamos r1 = r2 = 0.5 en todas las actualizaciones.

```
Estado inicial:
P1: x=4,  v=0,  p₁=4      P2: x=−2, v=0,  p₂=−2
f(4)=16, f(−2)=4  →  g = −2

Iteración 1:
P1: v = 0.5·0 + 0.5·(4−4) + 0.5·(−2−4) = −3         x = 4−3 = 1      f=1
P2: v = 0.5·0 + 0.5·(−2+2) + 0.5·(−2+2) = 0          x = −2           f=4
Actualizar memorias: p₁=1 (f=1 < 16), g = 1 (f=1 < 4)

Iteración 2:
P1: v = 0.5·(−3) + 0.5·(1−1) + 0.5·(1−1) = −1.5      x = 1−1.5 = −0.5 f=0.25
P2: v = 0.5·0 + 0.5·(−2+2) + 0.5·(1+2) = 1.5          x = −0.5         f=0.25
p₁ = −0.5, g = −0.5;  p₂ = −0.5

Iteración 3:
P1: v = 0.5·(−1.5) + 0 + 0 = −0.75                    x = −1.25        f=1.56 (peor: no actualiza p₁)
P2: v = 0.5·1.5 + 0 + 0 = 0.75                        x = 0.25         f=0.0625 → g = 0.25
```

En 3 iteraciones el mejor global pasó de f = 4 a f = 0.0625, oscilando alrededor del óptimo x = 0 — el comportamiento típico de PSO: sobrepasar y volver, con amplitud amortiguada por w < 1. Nótese que P1 empeoró en la iteración 3: las partículas individuales fluctúan; el progreso se mide en `g`.

Intuición ACO (mini-TSP de 4 ciudades): con τ inicial uniforme, la primera iteración elige casi por η (distancias); si una hormiga encuentra el tour corto A-B-D-C, sus aristas reciben $\Delta\tau = Q/L$ mayor; en la siguiente iteración P(A→B) crece, más hormigas lo prueban y lo refinan. La evaporación ρ impide que un tour mediocre temprano se fosilice.

📊 Propiedades y comparación

Aspecto	PSO	ACO	GA (033)
Dominio natural	Continuo	Combinatorio (grafos)	Ambos (según codificación)
Memoria	p_i y g (posiciones)	Matriz de feromonas	Población misma
Comunicación	Directa (broadcast de g)	Estigmergia (entorno)	Cruce entre pares
Operadores	Suma vectorial	Construcción probabilística	Selección/cruce/mutación
Hiperparámetros clave	w, c_1, c_2	α, β, ρ, m	N, pc, pm, presión
Modo de fallo	Colapso prematuro en g	Congelación de feromona	Convergencia prematura
Garantías	Ninguna práctica	Convergencia probabilística (variantes)	Ninguna práctica

⚠️ Errores conceptuales frecuentes

1. **"El enjambre es inteligente porque los agentes lo son."** Al revés: los agentes son triviales; la inteligencia emerge de la interacción y la memoria compartida. No hay controlador central que optimice.
2. **Juzgar PSO por la trayectoria de una partícula.** Las partículas individuales oscilan y empeoran; el objeto que converge es el mejor global g (como en el ejemplo trabajado).
3. **Olvidar la evaporación en ACO.** Con ρ = 0 la feromona solo crece y la primera ruta decente monopoliza la búsqueda: retroalimentación positiva sin freno = estancamiento garantizado.
4. **Trasplantar PSO a combinatoria (o ACO a continuo) sin rediseño.** La resta de posiciones no tiene sentido en permutaciones; hay variantes específicas, pero no es "gratis".
5. **Ignorar No Free Lunch.** Ni PSO ni ACO son "mejores que GA" en general; la comparación válida es empírica, en la familia de problemas concreta y con igual presupuesto de evaluaciones.

🚀 Del aprendizaje a la operación

El ejemplo usa 2 partículas y r fijos; en uso real: decenas-cientos de partículas/hormigas, r aleatorios con semillas registradas, criterios de parada por estancamiento, y sintonización de w/c o α/β/ρ que puede dominar el resultado. Para producción faltan además paralelización de evaluaciones (el costo real suele estar en f), manejo de restricciones, y reporte estadístico sobre múltiples corridas — un único run exitoso no es evidencia de nada en optimización estocástica.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("optimization")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kennedy, J. & Eberhart, R. (1995). "Particle swarm optimization". *Proc. IEEE ICNN*, 1942-1948. https://doi.org/10.1109/ICNN.1995.488968 — uso: fuente primaria del mecanismo estudiado
- Dorigo, M., Maniezzo, V. & Colorni, A. (1996). "Ant System: optimization by a colony of cooperating agents". *IEEE Trans. SMC-B*, 26(1), 29-41. https://doi.org/10.1109/3477.484436 — uso: fuente primaria del mecanismo estudiado
- Dorigo, M. & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press. https://mitpress.mit.edu/9780262042192/ant-colony-optimization/ — uso: referencia consultada en su fuente original
- Clerc, M. & Kennedy, J. (2002). "The particle swarm — explosion, stability, and convergence in a multidimensional complex space". *IEEE Trans. Evolutionary Computation*, 6(1), 58-73.

https://doi.org/10.1109/4235.985692 — uso: fuente primaria del mecanismo estudiado

- Bonabeau, E., Dorigo, M. & Theraulaz, G. (1999). *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P92 · Optimización por enjambre de partículas	1995	Optimiza sin gradiente con dos únicas memorias: lo mejor que ha encontrado cada individuo y lo mejor que ha encontrado el grupo.	notebook
P93 · Sistema de hormigas: optimización mediante una colonia de agentes cooperantes	1996	La solución no está en ningún agente: está en el rastro que dejan en el entorno y que se refuerza y se evapora.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Dorigo, Marco y Stützle, Thomas — *Ant Colony Optimization (Bradford Books)*	2004	ISBN 9780262042192 · web de la obra	citada en las referencias de esta clase
Bonabeau, E., Dorigo, M. y Theraulaz, G. — *Swarm Intelligence: From Natural to Artificial Systems*	1999	*sin localizador verificado*	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 02 · bloque de incertidumbre y decisión
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	obra de referencia de la parte 02 · modelos gráficos probabilísticos
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define optimización por enjambre y colonia sin usar una marca o framework como definición.
2. Explica la relación entre PSO, ACO, metaheurística, exploración.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

035 — Programación probabilística y causalidad

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 6
Laboratorio: `probability` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **programación probabilística y causalidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar programación probabilística y causalidad usando los conceptos `causalidad`, `intervención`, `modelos`, `inferencia`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`causalidad`, `intervención`, `modelos`, `inferencia`

🗺️ Ubicación en el mapa de la IA

Esta clase reúne dos culminaciones de la parte 02. La **programación probabilística** generaliza las redes bayesianas (027): el modelo es un programa con variables aleatorias, y un motor de inferencia (MCMC, variacional — clase 031) calcula posteriores automáticamente. La **causalidad** de Pearl responde a la limitación más profunda de todo lo anterior: la probabilidad condicional describe qué *observamos*, no qué pasaría si *interviniéramos*. Correlación no responde preguntas de acción; el do-calculus sí, y es la frontera actual entre predecir y entender.

📖 Fundamentos

💻 Programación probabilística (PPL)

Un programa probabilístico define un modelo generativo con dos primitivas:

```
# pseudocódigo estilo PPL
p    = sample("p", Beta(1, 1))          # prior sobre variable latente
for i in range(N):
    observe(f"y{i}", Bernoulli(p), y[i])  # condicionar en datos
# consulta: posterior de p dado y
```

El motor (PyMC, Stan, NumPyro, Gen) separa **modelo** de **inferencia**: el usuario declara la historia generativa; el sistema corre MCMC/variacional. Ventajas sobre redes dibujadas a mano: control de flujo, recursión, número variable de variables. Costo: la inferencia automática puede fallar silenciosamente (cadenas que no convergen), y hay que auditarla con diagnósticos ($\hat{R}$, tamaño efectivo).

🪜 La escalera de la causalidad (Pearl)

```
Nivel 1 — Asociación:     P(y | x)        "¿qué veo si observo x?"        (datos)
Nivel 2 — Intervención:   P(y | do(x))    "¿qué pasa si HAGO x?"          (experimento o do-calculus)
Nivel 3 — Contrafactual:  P(y_x | x', y') "¿qué habría pasado si…?"       (modelo estructural completo)
```

`P(y | x) ≠ P(y | do(x))` en general: observar zapatos grandes predice buena lectura en niños (asociación vía edad), pero regalar zapatos no enseña a leer (intervención). El operador `do(X=x)` corta las flechas que entran en X — X deja de escuchar a sus causas — y modela una acción externa.

🔀 Confusión (confounding) y el criterio backdoor

Un **confounder** Z causa a la vez el tratamiento X y el resultado Y, creando asociación espuria. Un conjunto Z satisface el **criterio backdoor** para (X, Y) si: (1) ningún nodo de Z es descendiente de X, y (2) Z bloquea (d-separación, clase 027) todo camino entre X e Y que entra a X por una flecha "hacia atrás". Entonces:

```
Fórmula de ajuste:
P(y | do(x)) = Σ_z P(y | x, z) · P(z)
```

— se estratifica por el confounder y se promedia con los pesos *poblacionales* de Z (no los condicionales a X). El **do-calculus** (tres reglas de reescritura sobre el grafo) generaliza esto y es completo: si el efecto causal es identificable desde datos observacionales + grafo, las reglas lo derivan.

Advertencia dual: ajustar por un **colisionador** o por un **mediador** introduce sesgo en lugar de quitarlo. Controlar "por todo lo que se pueda" es un error causal, no prudencia estadística.

🧩 Modelos causales estructurales (SCM)

Un SCM asigna a cada variable una ecuación $X_i := f_i(\text{Padres}(X_i), U_i)$ con ruidos exógenos U. El `:=` es asimétrico (mecanismo, no ecuación algebraica). Los SCM soportan los tres niveles: simular (1), mutilar el grafo con do (2), y abducir los U para razonar contrafactualmente (3). Un PPL puede *expresar* un SCM — la síntesis de las dos mitades de esta clase.

🧮 Ejemplo trabajado

¿El tratamiento X cura la enfermedad Y? Confounder: gravedad Z (los graves reciben el tratamiento con más frecuencia y se curan menos). Datos observacionales de 1 000 pacientes:

Z (gravedad)	P(z)	P(x=1 | z)	P(y=1 | x=1, z)	P(y=1 | x=0, z)
leve (z_0)	0.5	0.2	0.9	0.8
grave (z_1)	0.5	0.8	0.6	0.4

Nivel 1 — asociación cruda (promedios ponderados por quién recibe el tratamiento):

```
P(y=1 | x=1) = [0.5·0.2·0.9 + 0.5·0.8·0.6] / (0.5·0.2 + 0.5·0.8)
             = (0.09 + 0.24) / 0.5 = 0.66
P(y=1 | x=0) = [0.5·0.8·0.8 + 0.5·0.2·0.4] / (0.5·0.8 + 0.5·0.2)
             = (0.32 + 0.04) / 0.5 = 0.72
Diferencia observacional: 0.66 − 0.72 = −0.06  →  ¡el tratamiento "empeora"!
```

Nivel 2 — intervención con ajuste backdoor (Z bloquea el camino trasero $X \leftarrow Z \rightarrow Y$):

```
P(y=1 | do(x=1)) = Σ_z P(y=1|x=1,z)·P(z) = 0.9·0.5 + 0.6·0.5 = 0.75
P(y=1 | do(x=0)) = 0.8·0.5 + 0.4·0.5 = 0.60
Efecto causal medio (ATE): 0.75 − 0.60 = +0.15  →  el tratamiento CURA
```

La asociación cruda (−0.06) y el efecto causal (+0.15) tienen **signo opuesto** — paradoja de Simpson resuelta: el grafo dice que Z es confounder y hay que ajustar; sin grafo, los mismos números no deciden si estratificar o no.

📊 Propiedades y comparación

Pregunta	Herramienta suficiente	Requiere	Falla si
P(y|x) — predicción	Estadística / ML estándar	Datos i.i.d.	Deriva de distribución
P(y|do(x)) — efecto de acción	RCT (experimento)	Aleatorización costosa/ética	No siempre posible
P(y|do(x)) sin experimento	do-calculus + grafo	DAG causal correcto y completo	Confounders no medidos
Contrafactual individual	SCM completo	Ecuaciones y ruidos	Casi nunca identificable solo con datos
Posterior de parámetros	PPL + MCMC	Modelo generativo	Cadenas sin converger

⚠️ Errores conceptuales frecuentes

1. `P(y|x)` **como efecto de una acción.** Condicionar selecciona subpoblaciones; intervenir cambia el mecanismo. El ejemplo trabajado muestra que pueden tener signo contrario.
2. **"Controlar por todas las variables disponibles."** Ajustar por colisionadores o mediadores *crea* sesgo (sesgo de colisionador, sobreajuste del efecto); la selección de covariables es un problema del grafo, no de cantidad.
3. **Creer que el DAG sale de los datos.** De datos observacionales solo se identifica la clase de equivalencia de Markov; la orientación causal exige supuestos, experimentos o conocimiento del dominio.
4. **Confundir PPL con "el modelo es correcto".** El motor calcula el posterior *del modelo escrito*; si la historia generativa es errónea, la inferencia impecable da respuestas precisas a la pregunta equivocada. Diagnósticos ≠ validación del modelo.
5. **Saltar al nivel 3 sin fundamento.** Los contrafactuales individuales ("¿se habría curado sin el fármaco?") requieren el SCM completo con sus ruidos; ni un RCT basta para identificarlos en general.

🚀 Del aprendizaje a la operación

El ejemplo tiene un confounder medido y un grafo dado; en la práctica el grafo es hipótesis discutible, hay confounders no medidos (se necesitan variables instrumentales, front-door o análisis de sensibilidad) y los datos tienen sesgos de selección propios. Un pipeline causal serio incluye: elicitar y documentar el DAG con expertos, verificar sus implicaciones testeables (independencias), estimar con métodos dobles-robustos y reportar cuánto confounding oculto haría desaparecer el efecto (E-value). En PPL: diagnósticos de convergencia y validación predictiva posterior antes de creer un número.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("probability")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.

- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*, 2.ª ed. Cambridge University Press. https://bayes.cs.ucla.edu/BOOK-2K/ — uso: referencia consultada en su fuente original
- Pearl, J. (1995). "Causal diagrams for empirical research". *Biometrika*, 82(4), 669-688. https://doi.org/10.1093/biomet/82.4.669 — uso: fuente primaria del mecanismo estudiado
- Pearl, J. & Mackenzie, D. (2018). *The Book of Why*. Basic Books. — uso: desarrollo extendido del tema

- Koller, D. & Friedman, N. (2009). *Probabilistic Graphical Models*, cap. 21 (causalidad). https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/ — uso: referencia consultada en su fuente original
- Documentación de PyMC. https://www.pymc.io/ · Documentación de Stan. https://mc-stan.org/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P94 · Stan: un lenguaje de programación probabilística	2017	Separa declarar el modelo de calcular la inferencia: se escribe qué se supone del mundo y el motor devuelve la posterior.	notebook
P95 · Las siete herramientas de la inferencia causal, con reflexiones sobre aprendizaje automático	2019	Ordena en tres peldaños lo que un sistema puede responder —asociación, intervención y contrafáctico— y muestra que subir de peldaño exige supuestos que los datos no contienen.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	citada en las referencias de esta clase · cap. 21 · obra de referencia de la parte 02
Pearl, J. y Mackenzie, D. — *The Book of Why*	2018	ISBN 9780465097609	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	obra de referencia de la parte 02 · bloque de incertidumbre y decisión
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define programación probabilística y causalidad sin usar una marca o framework como definición.
2. Explica la relación entre causalidad, intervención, modelos, inferencia.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

036 — Proyecto: sistema híbrido para decisiones

Parte: 02 — IA probabilística, evolutiva y de decisión
Nivel: intermedio · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: sistema híbrido para decisiones** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: sistema híbrido para decisiones usando los conceptos `Bayes`, `reglas`, `optimización`, `trazabilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`Bayes`, `reglas`, `optimización`, `trazabilidad`

🗺️ Ubicación en el mapa de la IA

Esta clase cierra la Parte 02 integrando sus tres familias: la inferencia probabilística (025-028, 035), la decisión secuencial bajo incertidumbre (029-030) y la optimización sin gradientes (031-034). Un sistema híbrido de decisión combina una red bayesiana para estimar el estado del mundo, un criterio de utilidad esperada (o un MDP si la decisión es secuencial) para elegir la acción, y un optimizador evolutivo para ajustar los parámetros libres del diseño. Este patrón —percibir con probabilidad, decidir con utilidad, afinar con búsqueda— reaparece en la Parte 03 con modelos aprendidos de datos y en el aprendizaje por refuerzo posterior.

📖 Fundamentos

🧱 Anatomía de un sistema híbrido de decisión

Un sistema de decisión bajo incertidumbre se descompone en cuatro capas con contratos explícitos entre sí:

```
1. Capa de creencias (red bayesiana)
   entrada : evidencia observada e (síntomas, sensores, señales)
   salida  : posterior P(H | e) sobre los estados relevantes del mundo

2. Capa de decisión (utilidad esperada / MDP)
   entrada : P(H | e) + tabla de utilidades U(acción, estado)
   salida  : acción a* = argmax_a Σ_h P(h | e) · U(a, h)
             (si la decisión es secuencial: política π* de un MDP
              resuelta con value iteration)

3. Capa de optimización (algoritmo evolutivo)
   entrada : parámetros libres θ del sistema (umbrales de decisión,
             pesos de la utilidad, hiperparámetros)
   salida  : θ* que maximiza el desempeño esperado sobre escenarios
             simulados (Monte Carlo)

4. Capa de trazabilidad
   registra: evidencia usada, posterior calculado, utilidades,
             acción elegida, semilla y versión de parámetros.
```

La separación importa: las creencias son *epistémicas* (qué es probablemente cierto) y las utilidades son *normativas* (qué preferimos). Mezclarlas —por ejemplo, inflar una probabilidad porque el error sería caro— corrompe ambas capas; el costo del error pertenece a `U`, no a `P`.

🔗 Contratos entre capas

- **BN → decisión:** el posterior debe estar normalizado y calibrado; la capa de decisión lo consume tal cual, sin re-ponderarlo.
- **Decisión → optimización:** la función objetivo del optimizador es la utilidad esperada media sobre `N` escenarios muestreados (Monte Carlo, clase 031); la varianza del estimador (~1/√N) limita cuán finas pueden ser las comparaciones entre candidatos.
- **MDP dentro de la capa de decisión:** si las acciones tienen consecuencias en el tiempo, la acción óptima ya no es un `argmax` puntual sino la política de un MDP `(S, A, P, R, γ)` resuelta por iteración de valores (clase 029). El posterior de la BN alimenta el estado de creencia inicial.

📏 Criterios de diseño de sistemas de decisión

1. **Separar estimación de preferencia** (probabilidad vs utilidad).
2. **Explicitar el costo de cada error**: una matriz `U(a, h)` completa, no solo "acierto/fallo".
3. **Cuantificar la robustez**: análisis de sensibilidad — ¿la acción óptima cambia si el prior o una utilidad se mueve ±20 %? Si `a*` es estable en todo el rango plausible, la decisión es robusta; si no, el sistema debe pedir más evidencia o escalar a un humano.
4. **Reproducibilidad**: semilla, versión de parámetros y evidencia registradas en cada decisión.
5. **Umbral de abstención**: si `max_a EU(a)` supera al segundo mejor por menos que la incertidumbre del estimador, la salida honesta es "no decidir aún".

📉 Análisis de sensibilidad

Sea `EU(a; θ)` la utilidad esperada de la acción `a` con parámetros `θ` (priors, CPTs, utilidades). El análisis de sensibilidad de una vía perturba un parámetro a la vez dentro de su rango plausible y observa si `argmax_a EU(a; θ)` cambia. El **valor de cruce** es el valor del parámetro donde dos acciones empatan; si está lejos de la

estimación puntual, la decisión es insensible a ese parámetro. Esto convierte "el modelo puede estar mal" en una afirmación medible: *cuánto* tendría que estar mal para cambiar la recomendación.

🧮 Ejemplo trabajado

Sistema de triaje de mantenimiento: decidir si **detener** una máquina o **continuar** produciendo.

Capa 1 — creencias. Red de dos nodos: `Falla → Vibración`. Prior `P(Falla)=0.10`; sensor con `P(Vib | Falla)=0.9`, `P(Vib | ¬Falla)=0.2`. Se observa vibración:

```
P(Falla | Vib) = 0.9·0.10 / (0.9·0.10 + 0.2·0.90)
               = 0.09 / (0.09 + 0.18) = 0.333
```

Capa 2 — decisión. Utilidades (en miles): `U(detener, falla)=−5` (reparación programada), `U(detener, ¬falla)=−5` (parada innecesaria cuesta lo mismo), `U(continuar, falla)=−40` (rotura catastrófica), `U(continuar, ¬falla)=0`.

```
EU(detener)   = 0.333·(−5)  + 0.667·(−5)  = −5.0
EU(continuar) = 0.333·(−40) + 0.667·0     = −13.3
→ a* = detener, aunque P(falla) sea solo 1/3: el costo asimétrico decide.
```

Capa 3 — sensibilidad. ¿Con qué posterior empatan? `−5 = p·(−40)` → `p = 0.125`. La recomendación "detener" se mantiene mientras `P(Falla | Vib) > 0.125`; el posterior estimado (0.333) está lejos del cruce, así que la decisión es robusta a errores moderados del prior. Despejando hacia el prior: el empate ocurre con `P(Falla) ≈ 0.031` — habría que creer que las fallas son 3 veces más raras de lo estimado para cambiar de acción.

Capa 4 — optimización. Si el umbral de alarma del sensor es un parámetro continuo `θ`, un algoritmo evolutivo (clase 033) busca el `θ*` que maximiza la utilidad media sobre miles de escenarios simulados con Monte Carlo, sin necesitar gradientes del simulador.

📊 Propiedades y comparación

Arquitectura	Estado del mundo	Preferencias	Secuencialidad	Ajuste de parámetros	Riesgo principal
Reglas duras (`si vib → detener`)	Implícito, binario	Implícitas en la regla	No	Manual	Umbrales arbitrarios, sin grados
Solo red bayesiana	Posterior explícito	Ausentes	No	CPTs estimadas	Reporta creencia, no decide
BN + utilidad esperada (esta clase)	Posterior explícito	Matriz U explícita	No	Sensibilidad + evolución	U difícil de elicitar
MDP completo	Estado + transiciones	Recompensa por paso	Sí (política)	Value iteration	Explosión de estados
RL aprendido de datos	Implícito en la red	Recompensa	Sí	Gradientes/simulación	Opacidad, necesita muchos datos

⚠️ Errores conceptuales frecuentes

1. **"La acción óptima es la del estado más probable."** Falso: con `P(falla)=0.33` el estado más probable es `¬falla`, pero la acción óptima es detener. El `argmax` se toma sobre utilidades esperadas, no sobre probabilidades.
2. **Ajustar probabilidades para reflejar costos.** Inflar `P(falla)` "por precaución" rompe la calibración y hace imposible auditar el sistema; la precaución se codifica en `U`.
3. **Optimizar sin controlar el ruido de Monte Carlo.** Comparar dos candidatos `θ` con pocas simulaciones selecciona ganadores por azar; hay que fijar semillas comunes (common random numbers) o aumentar `N` hasta que la diferencia supere el error estándar.
4. **Presentar la recomendación sin sensibilidad.** Un `a*` sin valores de cruce es una caja negra: no se sabe si la decisión pende de un prior dudoso.
5. **Confundir demo con despliegue.** Este proyecto valida el patrón de integración; un sistema real exige CPTs estimadas de datos, elicitación formal de utilidades y revisión humana proporcional al riesgo.

🚀 Del aprendizaje a la operación

El capstone integra motores didácticos con parámetros dados. En operación real faltan: (1) estimar las CPTs desde datos históricos con incertidumbre de muestreo (y re-estimarlas ante deriva); (2) elicitar utilidades con las partes interesadas mediante técnicas formales (loterías de referencia), no números inventados; (3) validar la calibración del posterior antes de conectarlo a decisiones; (4) definir gobernanza — quién revisa las decisiones de alto impacto y cómo se audita la traza; (5) monitoreo continuo del desempeño de la política frente al baseline humano.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell & Norvig — *Artificial Intelligence: A Modern Approach*, 4e (caps. 13, 16 y 17: incertidumbre, decisiones simples y decisiones secuenciales) — uso: desarrollo extendido del tema
- Koller & Friedman — *Probabilistic Graphical Models: Principles and Techniques* (MIT Press) — uso: referencia consultada en su fuente original
- Sutton & Barto — *Reinforcement Learning: An Introduction*, 2e (cap. 3-4: MDP y programación dinámica; PDF oficial gratuito) — uso: desarrollo extendido del tema
- Howard, R. A. (1966) — "Information Value Theory", *IEEE Transactions on Systems Science and Cybernetics* (análisis de decisiones y valor de la información) — uso: fuente primaria del mecanismo estudiado
- Bellman, R. (1957) — "A Markovian Decision Process", *Journal of Mathematics and Mechanics* — uso: referencia consultada en su fuente original
- Kochenderfer, Wheeler & Wray — *Algorithms for Decision Making* (MIT Press, PDF oficial gratuito) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P95 · Las siete herramientas de la inferencia causal, con reflexiones sobre aprendizaje automático	2019	Ordena en tres peldaños lo que un sistema puede responder —asociación, intervención y contrafáctico— y muestra que subir de peldaño exige supuestos que los datos no contienen.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Koller, Daphne y Friedman, Nir — *Probabilistic Graphical Models: Principles and Techniques*	2010	ISBN 9780262013192 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 02
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 13 · obra de referencia de la parte 02
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · cap. 3-4
Pearl, J. — *Probabilistic Reasoning in Intelligent Systems*	1988	ISBN 9780080514895	obra de referencia de la parte 02 · redes de creencia

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: sistema híbrido para decisiones sin usar una marca o framework como definición.
2. Explica la relación entre Bayes, reglas, optimización, trazabilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
